

Escola Europeia de Ensino Profissional



Prova de Aptidão Profissional

Braço Robótico

2024 / 2025

7907 Lounie Costa

7921 Mário Moniz

Braço Robótico

Escola Europeia de Ensino Profissional

2024 / 2025

Técnico de Eletrónica, Automação e Computadores



30 abril de 2025

7907 Lounie Costa

7921 Mário Moniz

Orientador Prof. Moisés Rodrigues

“A winner is a dreamer who never gives up.”

Nelson Mandela





Agradecimentos

Gostaríamos de exprimir a nossa profunda gratidão à prestigiosa Escola Europeia de Ensino Profissional (EEEP) por nos proporcionar um ciclo extraordinário de três anos de estudos, de conhecimento e práticas enquadradas no curso de Técnico de Eletrónica, Automação e Computadores (TEAC).

Também gostaríamos de agradecer especialmente ao nosso orientador da Prova de Aptidão Profissional (PAP), Professor Moisés Rodrigues, que esteve sempre presente na ajuda necessária, promovendo o conhecimento durante os 3 anos de curso. Ajudou-nos e orientou-nos no desenvolvimento de cada projeto, disponibilizando algumas das suas aulas e o tempo necessário para nos ajudar na fase do desenvolvimento dos projetos. Esteve sempre disposto a tirar-nos dúvidas e no auxílio necessário, daí a nossa profunda gratidão.

Agradecemos também ao nosso Coordenador do Curso, Professor Sérgio Silva, que nos ajudou em todos os momentos que precisamos, partilhando muitos dos conhecimentos cruciais do curso.



Resumo

O projeto *Braço Robótico* visa a criação de um protótipo de um braço com um sistema de captação de movimentos em tempo real (Leap Motion) e replicação desse movimento no braço.

O principal objetivo na elaboração deste projeto é a possibilidade de aplicação do mesmo em escolas, podendo ser utilizado pelos professores como uma ferramenta de apoio ao ensino, fazendo analogia com a fisioterapia das mãos utilizada em contexto de saúde.

O projeto tem como propósito a replicação do movimento da mão, utilizando tecnologias de *traking* do movimento e o mapeamento para funcionar em tempo real e replicando movimentos precisos de acordo com os movimentos das mãos.

Este projeto abrange diversas facetas na sua criação, oferecendo uma solução completa e eficaz. A dificuldade em simplificar tarefas mostra a necessidade de tornar o ambiente profissional mais prático e eficiente.

Palavras-Chave: *Leap motion, inmoov, estrutura robótica 3d, braço robótico*



Índice Geral

Agradecimentos.....	iv
Resumo.....	v
Índice Geral	vi
Índice de Figuras	ix
Índice de Tabelas	x
Índice de Codificação	xi
Notação e Glossário	xii
1 Introdução.....	1
1.1 Motivação	1
1.2 Apresentação do projeto.....	1
1.3 Metodologias e Estratégias	2
1.4 Organização do relatório	1
2 Planificação da PAP.....	2
2.1 Fases do Projeto	2
2.1.1 Mapa Mental	2
2.1.2 Diagrama Temporal	3
2.1.3 Diagrama de Funcionamento	4
2.1.4 Diagrama de Atividades	5
2.2 Recursos de Hardware	6
2.3 Recursos de Software.....	7
2.4 Recursos Humanos e Físicos	8
2.5 Orçamento	8
2.6 Público-alvo.....	9



3	Descrição técnica.....	10
3.1	Fase 1 – Desenvolvimento do Circuito Eletrónico	10
3.1.1	Definição das tecnologias	12
3.1.2	Hardware	14
3.1.3	Projeção do circuito elétrico.....	17
3.1.4	Criação do desenho 3D	18
3.1.5	Instalação do Software	20
3.2	Fase 2 – Impressão 3D das Peças do Braço Robótico.....	22
3.2.1	Preparação dos ficheiros de impressão 3D das peças do braço	22
3.2.2	Impressão das peças utilizando filamento adequado	23
3.3	Fase 3 – Montagem Mecânica	24
3.4	Fase 4 – Montagem do Circuito Eletrónico	27
3.4.1	Montagem e ligação de todos os componentes eletrónicos.....	27
3.4.2	Teste de todas as ligações	29
3.5	Fase 5 – Programação do Microcontrolador Arduino.....	30
3.5.1	Aquisição de dados	30
3.5.2	Controlo dos movimentos do braço	34
3.5.3	Teste e aprimoramento da programação	36
3.6	Fase 6 – Testes e Ajustes finais	38
3.6.1	Testar o funcionamento completo do braço robótico	38
3.6.2	Realizar ajustes necessários na montagem e programação	39
4	Conclusões	40
4.1	Objetivos realizados.....	40
4.2	Limitações e trabalho futuro	41
4.3	Apreciação final	41



Braço Robótico

IMP.189-03

4.4	Autoavaliação	42
4.4.1	Lounie Costa	42
4.4.2	Mário Moniz	42
4.5	Formação em Contexto de Trabalho	43
4.5.1	Lounie Costa	43
4.5.2	Mário Moniz	44
5	Bibliografia	45
6	Anexos	46
A.	Código	46
A1.	Arduino	46
A2.	Processing	47



Índice de Figuras

Figura 1 – Mapa Mental	2
Figura 2 – Diagrama Temporal do Projeto	3
Figura 3 – Diagrama de funcionamento	4
Figura 4 – Diagrama de atividades	5
Figura 5 – Estrutura 3D do projeto	11
Figura 6 – Arduino IDE	12
Figura 7 – MS Word	12
Figura 8 – PowerPoint	13
Figura 9 – Tinkercad	13
Figura 10 – Monday	13
Figura 12 – Processing	14
Figura 13 – Arduino Uno R3	14
Figura 14 – Servo Motor	15
Figura 15 – Braço Robótico	15
Figura 16 – Jumper	16
Figura 17 – Fish Line	16
Figura 18 – Leap Motion	16
Figura 19 – Circuito do Braço	17
Figura 20 – Circuito do braço com o Leap Motion	17
Figura 21 – Braço Robótico 3D	18
Figura 22 – Braço 3D	18
Figura 23 – Medidas do braço 3D	19
Figura 24 – Acesso ao site oficial do Processing	20
Figura 25 – Extração da Pasta Zipada	20
Figura 26 – Instalação do Arduino	21



Braço Robótico

IMP.189-03

Figura 27 – Configuração da localização da aplicação	21
Figura 28 – Aquisição dos STL	22
Figura 29 – Pedido de Impressão	23
Figura 30 – Envio do STL	23
Figura 31 – Perspetiva do antebraço	24
Figura 32 – Montagem antebraço	24
Figura 33 – Montagem dos Servos	25
Figura 34 – Montagem da Mão	25
Figura 35 – Acabamento	26
Figura 36 – Finalização da montagem do Braço	26
Figura 37 – Montagem do circuito	27
Figura 38 – Ligação no protótipo	28
Figura 39 – Teste das ligações	28
Figura 40 – Ligação Arduino	29
Figura 41 – Ligação servo motor	29
Figura 42 – Teste no Arduino	36
Figura 43 – Teste do programa	37
Figura 44 – Teste do Processing	37
Figura 45 – Teste no braço completo	38
Figura 46 – Ajustes necessários	39

Índice de Tabelas

Tabela 1- Recursos Hardware	6
Tabela 2-Recursos Software	7
Tabela 3-Orçamento	8



Índice de Codificação

<i>Código 1 – Bibliotecas.....</i>	<i>30</i>
<i>Código 2 – Variáveis</i>	<i>30</i>
<i>Código 3 – Void Setup.....</i>	<i>30</i>
<i>Código 4 – Listbox.....</i>	<i>31</i>
<i>Código 5 – Listagem serial port</i>	<i>31</i>
<i>Código 6 – Iniciar porta</i>	<i>31</i>
<i>Código 7 – Void draw.....</i>	<i>32</i>
<i>Código 8 – Cálculo do Ângulo</i>	<i>32</i>
<i>Código 9 – Enviar Ângulo.....</i>	<i>33</i>
<i>Código 10 – Início da função.....</i>	<i>33</i>
<i>Código 11 – Enviar para o Arduino</i>	<i>34</i>
<i>Código 12 – Bibliotecas.....</i>	<i>34</i>
<i>Código 13 – PinosPWM</i>	<i>34</i>
<i>Código 14 – Void Setup Arduino</i>	<i>35</i>
<i>Código 15 – Anexar pinos</i>	<i>35</i>
<i>Código 16 – Função ler dados.....</i>	<i>35</i>
<i>Código 17 – Execução no servo motor.....</i>	<i>36</i>
<i>Código 18 – Ajustes Necessários.....</i>	<i>39</i>



Notação e Glossário

CPU	<i>Central Processing Unit</i>
EEEP	<i>Escola Europeia de Ensino Profissional</i>
FCT	<i>Formação em Contexto de Trabalho</i>
GUI	<i>Graphical User Interface</i>
IDE	<i>Integrated Development Environment</i>
MS	<i>Microsoft</i>
PAP	<i>Prova de Aptidão Profissional</i>
PC	<i>Personal Computer</i>
PLA	<i>Polylactic Acid – Tipo de filamento utilizado na impressão 3D</i>
PWM	<i>Pulse Width Modulation</i>
STL	<i>Stereolithography File – Formato de ficheiro para impressão 3D</i>
TEAC	<i>Técnico de Eletrónica, Automação e Computadores</i>
USB	<i>Universal Serial Bus</i>



1 Introdução

1.1 Motivação

O motivo da escolha do tema para este projeto deve-se ao fato da sua possível utilidade em contexto escolar, em particular com alunos que apresentam alguma deficiência nas mãos, podendo ser utilizado pelos professores como uma ferramenta de apoio ao ensino. O projeto envolve distintas áreas do curso, sobretudo a programação e eletrónica.

1.2 Apresentação do projeto

O projeto da PAP consiste no desenvolvimento e construção de um Braço Robótico funcional, com foco na simulação de movimentos de uma mão humana. O projeto é concretizado com recurso à impressão 3D, programação de um microcontrolador, e integração de servomotores, proporcionando uma aplicação prática dos conhecimentos adquiridos ao longo do curso.

Os principais objetivos passam pela projeção e construção de um braço robótico funcional utilizando tecnologias acessíveis; aplicação dos conhecimentos de eletrónica, programação e automação; simulação dos movimentos básicos de uma mão humana, como abrir e fechar os dedos e estabelecer a comunicação entre um computador e o microcontrolador para controlo remoto dos movimentos.

Os resultados esperados com este projeto é um braço robótico com capacidade de realizar movimentos básicos dos dedos, a comunicação estável entre o computador e microcontrolador e a demonstração funcional do projeto, em ambiente controlado.



1.3 Metodologias e Estratégias

Para o planeamento e gestão do projeto, foram utilizadas estratégias e metodologias:

- Definição do orçamento e prazos para cada uma das fases.
- Identificação dos componentes e materiais necessários.
- Desenvolvimento do Circuito Eletrónico:
 - o Projeção do circuito elétrico para ligar os motores, sensores e o microcontrolador.
 - o Utilização de *software* de simulação para testar o circuito antes da montagem física.
- Impressão 3D das Peças do Braço Robótico:
 - o Preparação dos ficheiros de impressão 3D das peças do braço.
 - o Impressão das peças utilizando filamento adequado (ex.: PLA).
- Montagem Mecânica:
 - o Montagem de todas as peças impressas para criar o braço robótico funcional.
- Montagem do Circuito Eletrónico:
 - o Montagem e ligação de todos os componentes eletrónicos (motores, sensores e microcontroladores) de acordo com o circuito planeado.
 - o Garantir que todas as ligações estão seguras e funcionais.



- Programação do Microcontrolador Arduino:
 - o Programação do Arduino para controlar os movimentos do braço, definindo as tarefas que este irá executar.
 - o Teste e aprimoração da programação de acordo com as necessidades e os testes funcionais realizados.
- Testes e Ajustes Finais:
 - o Teste do funcionamento completo do braço robótico.
 - o Realização dos ajustes necessários na montagem e programação para garantir um desempenho eficiente.

1.4 Organização do relatório

O relatório está dividido em seis capítulos principais. O primeiro capítulo corresponde à Introdução, onde se abordam a motivação, os objetivos, as metodologias utilizadas e a estrutura do relatório. O segundo capítulo abrange Planificação do projeto, onde são detalhadas as fases de desenvolvimento, os recursos necessários (hardware, software, equipa e materiais), o orçamento e o público-alvo. O terceiro capítulo engloba uma descrição técnica do projeto, organizada em várias etapas, desde o desenvolvimento do circuito, a impressão 3D das peças, a montagem mecânica e eletrónica, a programação dos microcontroladores, os testes e ajustes finais. No quarto capítulo, são apresentadas as Conclusões, ressaltando os objetivos alcançados, as limitações encontradas, sugestões para trabalhos futuros, uma avaliação final e a autoavaliação dos autores da PAP. O quinto capítulo corresponde à Bibliografia utilizada, enquanto o sexto capítulo inclui os Anexos - código-fonte do projeto, fichas técnicas dos componentes e outros materiais complementares.

2 Planificação da PAP

2.1 Fases do Projeto

2.1.1 Mapa Mental

O mapa mental permite organizar as ideias do projeto. Nele encontram-se o tema do projeto - braço robótico composto por uma lista dos recursos que são utilizados: materiais necessários à construção do braço robótico, *hardware*, *software* e as linguagens de programação.

Desta forma, o mapa mental serve como uma ilustração gráfica e clara do projeto.

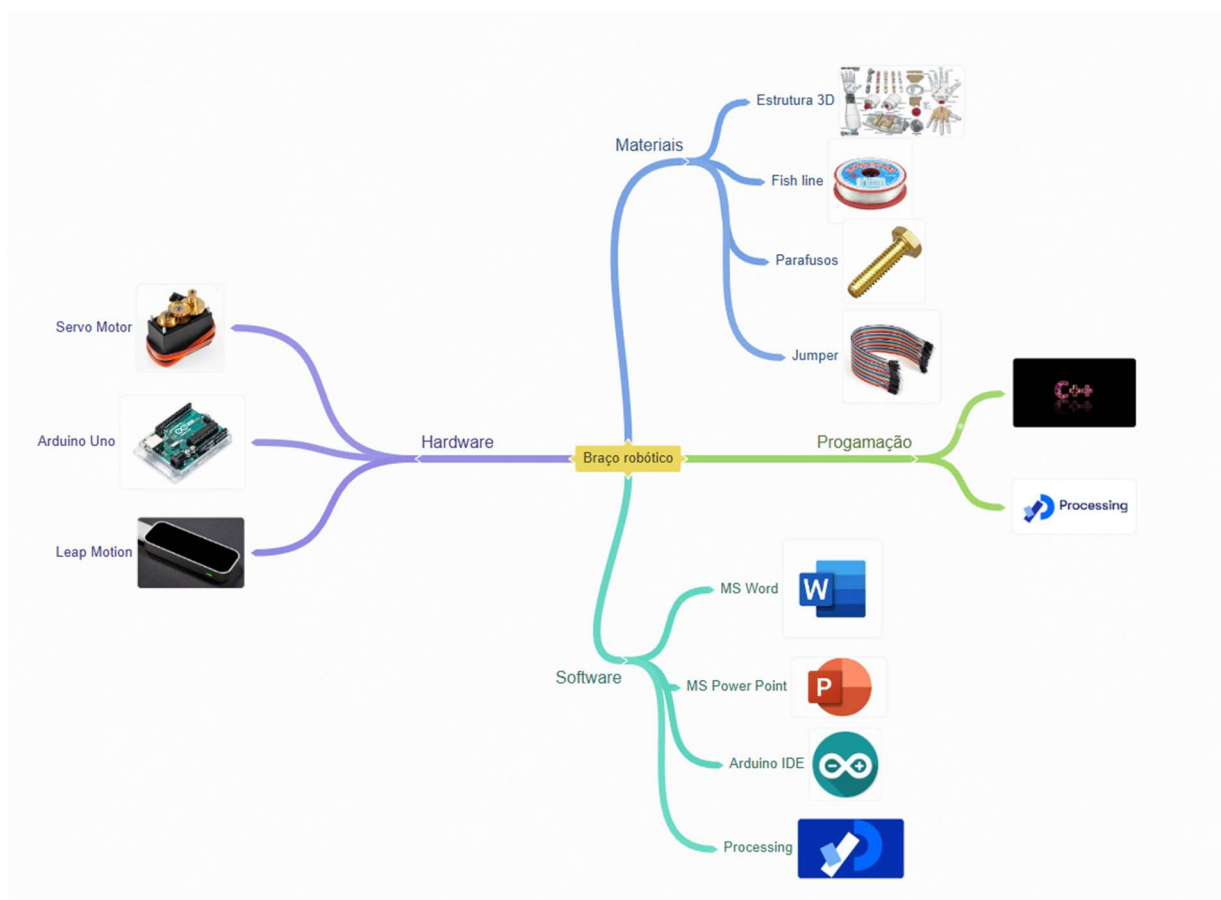


Figura 1 – Mapa Mental

2.1.2 Diagrama Temporal

O diagrama temporal estabelece temporalmente as distintas fases do projeto.



Figura 2 – Diagrama Temporal do Projeto

2.1.3 Diagrama de Funcionamento

Um diagrama de funcionamento é uma representação gráfica que mostra como os componentes do sistema interagem entre si para fazer o projeto funcionar. No projeto do braço robótico, o diagrama de funcionamento mostra o fluxo de informação entre os diferentes elementos (microcontrolador, sensores, servos, computador, etc.).

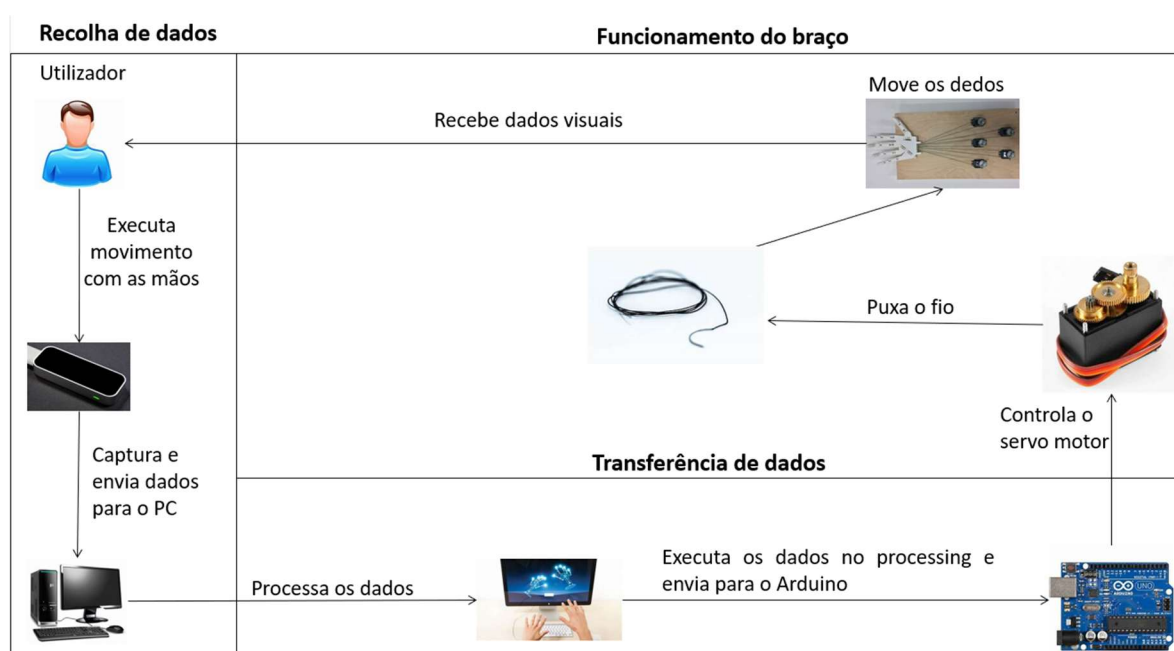


Figura 3 – Diagrama de funcionamento

Na Figura observa-se o funcionamento do circuito. O projeto inicia-se com o utilizador que executa movimento com as mãos que são captados pelo sensor. Posteriormente envia os dados para o computador que os processa e envia para o Arduino, que controla o servo motor responsável pelo movimento dos dedos do braço.

2.1.4 Diagrama de Atividades

O diagrama de atividades é utilizado para descrever a sequência de ações num processo. Mostra as atividades, decisões, paralelismos e ações que ocorrem durante a execução do projeto. Representa os passos necessários para realizar uma tarefa ou processo, como ligar o braço robótico, enviar um comando, ou realizar um movimento.

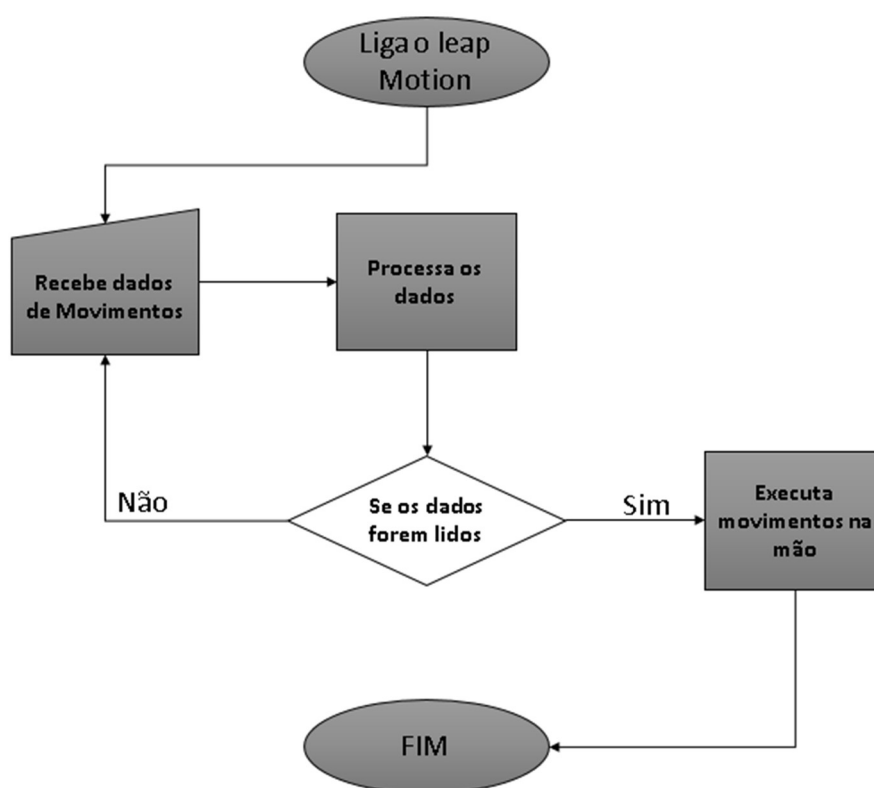


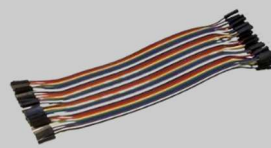
Figura 4 – Diagrama de atividades

A Figura acima mostra a diagrama de atividades principal do Projeto. Inicialmente, liga-se o *Leap Motion* que envia o sinal que é processado. Se os dados processados forem lidos corretamente, o circuito executa o movimento na mão, caso contrário, volta a processar os dados captados anteriormente.

2.2 Recursos de Hardware

Na Tabela 1 encontram-se enumerados todos os componentes de *hardware* utilizados na elaboração do projeto, incluindo os modelos e marcas correspondentes a cada componente, assim como a sua ilustração.

Tabela 1- Recursos Hardware

Componentes	Modelos/Marcas	Imagens
Arduino Uno R3	<i>Arduino</i>	
Servo Motor	<i>Mg996r</i>	
Sensor	<i>Leap Motion Controller</i>	
Estrutura do braço 3D	<i>Braço 3D</i>	
Fios Jumper	<i>Dupont Macho 150mm</i>	
Parafusos	<i>Diversos</i>	
Fish line	<i>Fish line 200LB</i>	

2.3 Recursos de Software

A tabela seguinte engloba os recursos de *software* utilizados, as suas versões utilizadas e uma breve ilustração dos mesmos.

Tabela 2-Recursos Software

Recursos De Software		
Software	Versão	Imagem
Arduino IDE	<i>Arduino IDE 2.3.3</i>	
Processing	<i>Processing 4.3</i>	
MS Word	<i>Word 2016</i>	
MS PowerPoint	<i>PowerPoint 2016</i>	
Monday	<i>Monday 5.19.3</i>	
Tinkercad	-----	

2.4 Recursos Humanos e Físicos

Para a elaboração deste projeto foram necessários recursos humanos e físicos.

Como *recursos humanos* contamos com a ajuda dos Professores:

- Moisés Rodrigues (Automação e Computadores) - auxílio da impressão das peças 3d para o braço robótico.
- Sérgio Silva (Sistemas Digitais e TIC) – auxílio na elaboração do relatório.

Nos *recursos físicos* foram necessárias as instalações: 1. Sala de aula (Sala 15) - Laboratório da EEEP: para a realização da Estrutura 3D. 2. Espaço de trabalho na habitação do aluno: para a elaboração da montagem do projeto.

2.5 Orçamento

A tabela seguinte apresenta o orçamento estimado para a realização do projeto, sendo que os valores apresentados encontram-se com a taxa de IVA em vigor.

Tabela 3-Orçamento

Orçamento		
Componente	Local de compra	Preço com IVA
Leap Motion	EBay	38€
Arduino UNO R3	KuantoKusta	19€
Impressão do braço	Mauser/Outros	234,82€
5 Servo Motores	Aliexpress	18.92€
Parafusos	Leroy Merlin	15€
Fios Jumper	Mauser	4,85€
Fish line	Outros	4,50€
Total		335,09 €

2.6 Público-alvo

Este projeto foi criado com o objetivo de atender às necessidades de um público específico, com um foco especial na educação inclusiva. O desenvolvimento visa beneficiar, principalmente, alunos com deficiência nas mãos, oferecendo uma ferramenta de apoio que pode ajudar no processo de aprendizagem de maneira prática e adaptada às suas limitações motoras.

Além dos alunos, o projeto também visa os professores do ensino básico, secundário/profissional e técnico, que poderão utilizar esta solução como um recurso didático complementar nas suas atividades pedagógicas. A ferramenta tem um grande potencial para ser integrada em práticas de ensino inclusivo, promovendo uma maior participação e envolvimento dos alunos com necessidades específicas.

Por fim, o projeto também é relevante para instituições de ensino profissional que abordam temas como programação, eletrónica, robótica e tecnologias assistidas. O seu desenvolvimento incorpora competências multidisciplinares adquiridas ao longo da formação, podendo servir como um exemplo prático da aplicação de conhecimentos teóricos em situações reais e socialmente significativas.

3 Descrição técnica

3.1 Fase 1 – Desenvolvimento do Circuito Eletrónico

O projeto consiste no movimento dos dedos de uma mão do braço robótico utilizando o servo motores, Arduino e o *Leap Motion*. O projeto tem como objetivo a replicação de movimentos executados pelo utilizador. Para a captação dos movimentos utiliza-se o *Leap Motion* que é um dispositivo que capta o movimento das mãos do utilizador. Utilizando um *software* específico, pode-se visualizar e retirar os dados necessários para o movimento no braço. Os dados recolhidos ilustram a quantos graus está a inclinação dos nossos dedos. Esses dados estão associados ao respetivo nome do dedo em questão.

O programa utilizado para reunir os dados fornecidos pelo *Leap Motion* é o *Processing* que é um *software* gráfico, gratuito e um ambiente de desenvolvimento integrado criado para projetos de eletrónica. Utilizando o *Processing* consegue-se replicar graficamente os movimentos do braço. Após a recolha, os dados são enviados para o Arduino. O Arduino está ligado aos servos motores que serão responsáveis pelos dedos. Existem 5 servos motores para os 5 dedos. O Arduino é responsável por fazer os servos motores funcionarem de acordo com os dados captados pelo *Leap Motion*. O Arduino recebe os dados do *Processing*, fazendo com que sejam executados pelo servo motor de acordo com os movimentos executados pelo utilizador. Esses movimentos servem para que os fios ligados entre o servo e a estrutura do braço sejam movidos de acordo com os dados recebidos pelo *Processing*.

Os servos motores utilizados no projeto são servos motores de 180 graus, fazendo com que os dedos possam completar perfeitamente os movimentos realizados pelo utilizador. São utilizados fios de pesca que são fios mais resistentes para suportar o peso e o desgaste será menor, quanto comparado com outros fios mais frágeis.

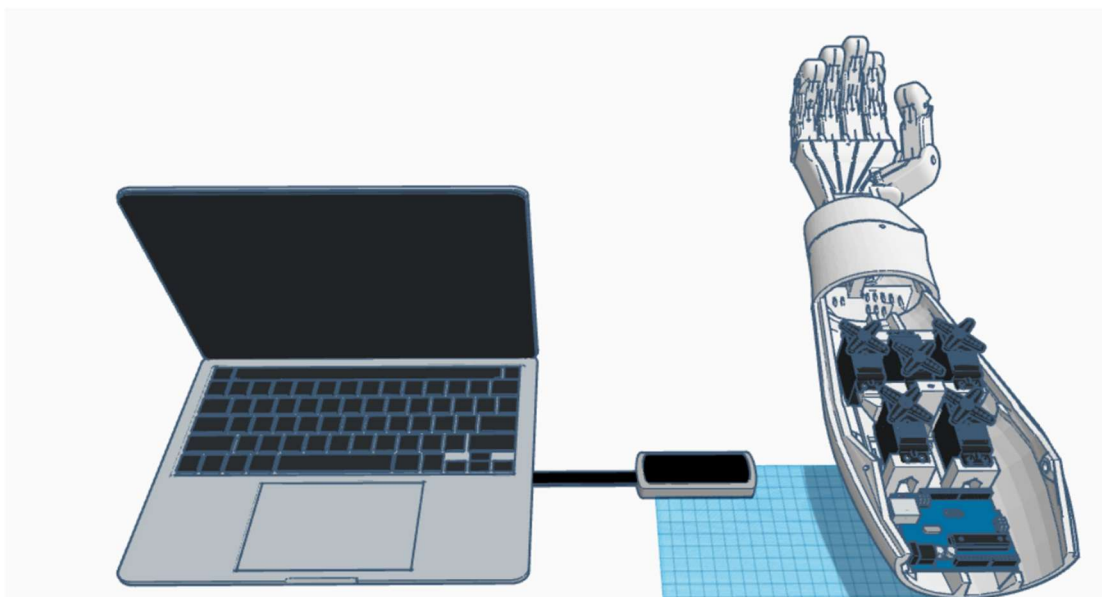


Figura 5 – Estrutura 3D do projeto

A imagem acima ilustra o projeto final após a sua conclusão. Como se observa, existe um computador e o *Leap Motion*. No computador está instalado o *software Processing* para que possam receber os dados do *Leap Motion*. Na estrutura 3D temos os servos motores e o Arduino. Desta forma, tem-se uma estrutura com menos exposições de fios e uma maior facilidade para a ligação do Arduino e os servos motores.

3.1.1 Definição das tecnologias

3.1.1.1 Software

O **Arduino IDE** é um *software* essencial no desenvolvimento de projetos com placas Arduino. Oferece uma interface que permite escrever, compilar e carregar código nas placas Arduino.

No contexto deste projeto, o Arduino IDE é usado para programar o Arduino Uno R3 que controla os servomotores.



Figura 6 – Arduino IDE

O **Microsoft Word** é uma ferramenta de processamento de texto que possibilita a criação, edição e formatação de documentos no computador.

No projeto, o Word foi utilizado para a elaboração do relatório da PAP.



Figura 7 – MS Word

O **Microsoft PowerPoint** é um *software* de criação e apresentação de *slides* que permite criar apresentações visuais dinâmicas.

O *PowerPoint* é utilizado para a elaboração de uma apresentação dinâmica que será utilizada como suporte no dia da defesa oral da PAP.



Figura 8 – PowerPoint

O **Tinkercad** é uma plataforma *online* de design 3D que permite a criação e edição de circuitos eletrónicos, sendo uma forma simples e intuitiva para a criação de modelos tridimensionais, a criação de circuitos. No contexto deste projeto, o Tinkercad utiliza-se para desenvolver a estrutura 3D do braço robótico e o circuito.



Figura 9 – Tinkercad

A **monday.com** é um sistema de trabalho que possibilita às equipas administrar facilmente projetos e fluxos de trabalho. No âmbito deste projeto o programa é utilizado para a criação do diagrama temporal.



Figura 10 – Monday

O **Processing** é um *software* que utiliza a linguagem Processing que é uma linguagem de programação de código aberto e ambiente de desenvolvimento integrado (IDE).

No projeto o Processing é utilizado para a captação de dados do *Leap Motion* e a sua projeção visual, além da comunicação com o Arduino.



Figura 11 – Processing

3.1.2 Hardware

O **Arduino Uno R3** é uma placa de microcontrolador baseada no chip ATmega 328P. A placa possui 14 pinos digitais de entrada/saída, dos quais 6 podem ser usados como saídas PWM, e 6 pinos analógicos.

O Arduino no projeto tem a função de controlar os servos motores, utilizando o *Leap Motion* para a captação de dados e enviá-los para o *Processing*. Uma comunicação entre os dois *software* (Processing, Arduino IDE), em que os dados são enviados para o Arduino que controla os servos motores.



Figura 12 – Arduino Uno R3

Um **servo motor** é um tipo de motor elétrico que é projetado para fornecer um controlo de posição preciso.

No projeto, o servo motor será controlado pelo Arduino para que faça mover os dedos utilizando os fios, de acordo com os dados recebidos do Arduino.



Figura 13 – Servo Motor

O **Braço Robótico** é uma estrutura criada por uma impressora 3D. Exemplifica um braço humano e serve como protótipo de um braço robótico. Nele são alocados todos os componentes para a composição do braço.



Figura 14 – Braço Robótico

Um **fio Jumper** é um cabo curto que é utilizado para fazer conexões elétricas entre componentes ou pinos em circuitos eletrónicos.

No projeto têm a função de fazer a ligação dos servos motores ao Arduino.



Figura 15 – Jumper

Fish Line são fios de pesca e terão a função de fazer as articulações da parte da mão.



Figura 16 – Fish Line

O **Leap Motion** consiste num pequeno dispositivo com um sensor capaz de captar movimentos dos 10 dedos das mãos do utilizador. No nosso projeto será utilizado para captar os movimentos dos dedos da mão direita e enviar os dados para os *Processing*.



Figura 17 – Leap Motion

3.1.3 Projeção do circuito elétrico

O circuito do projeto consiste numa ligação do Arduino e os servos Motores. O Arduino serve para a programação dos movimentos dos servos motores que se movem de acordo com os dados recebidos pelo *Leap Motion*. A quantidade dos servos são 5 demonstrando os dedos da mão. De acordo com o sinal recebido do Arduino, os servos movem-se de acordo com o grau de amplitude designado.

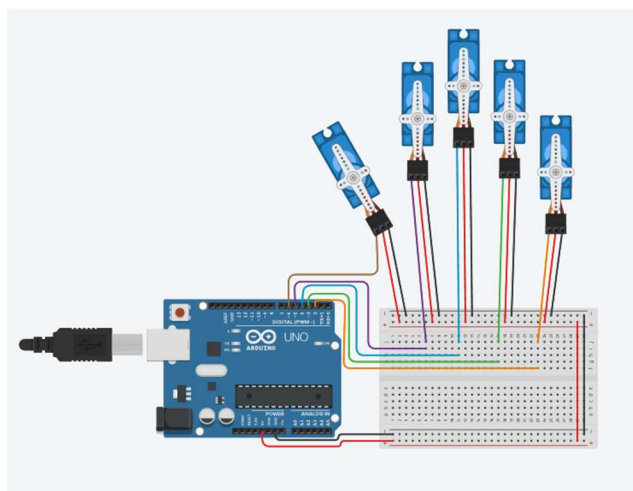


Figura 18 – Circuito do Braço

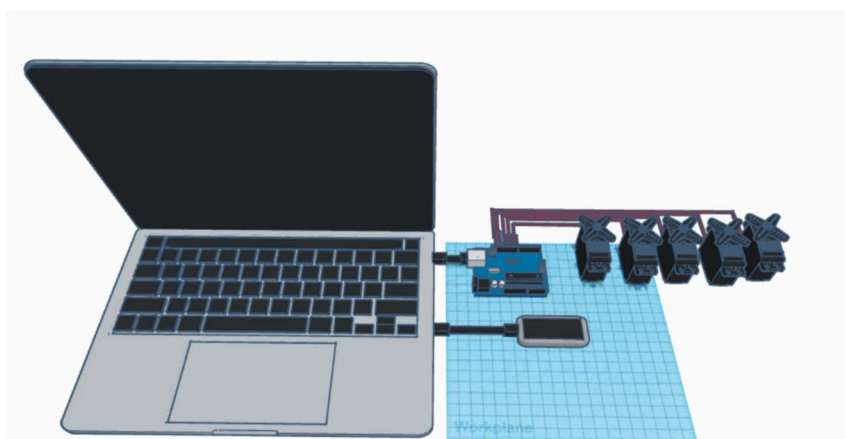


Figura 19 – Circuito do braço com o Leap Motion

3.1.4 Criação do desenho 3D

Num passo prévio à montagem física do braço robótico é necessário o desenho 3D através de um programa de *software* de simulação.

Na imagem seguinte apresenta-se o esquema 3D do braço robótico. É a representação final do braço, com os componentes já presentes. Assim, consegue-se ter uma ideia de como estará o projeto depois de finalizado.

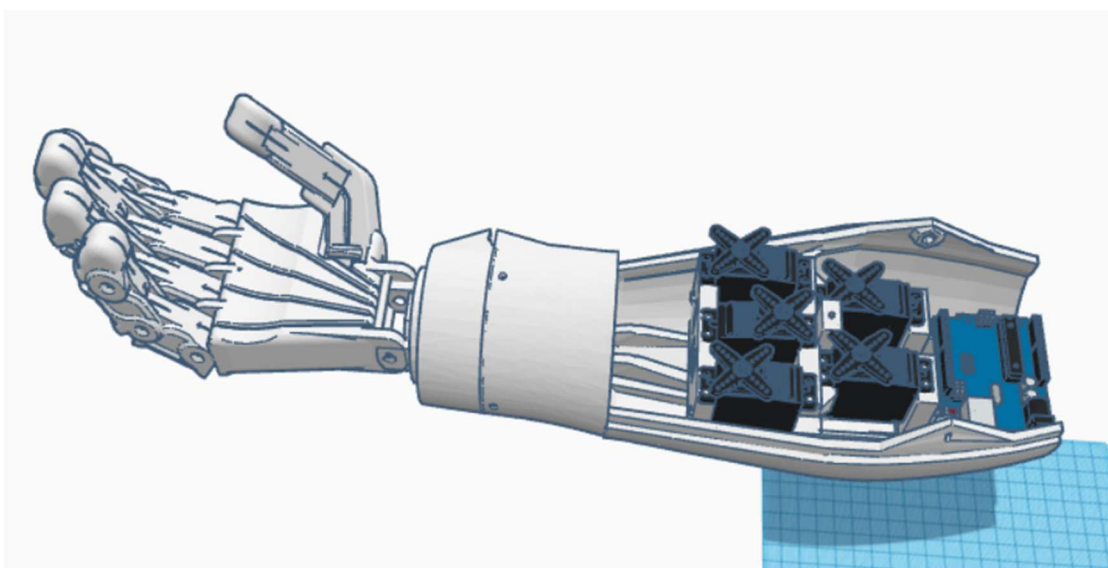


Figura 20 – Braço Robótico 3D

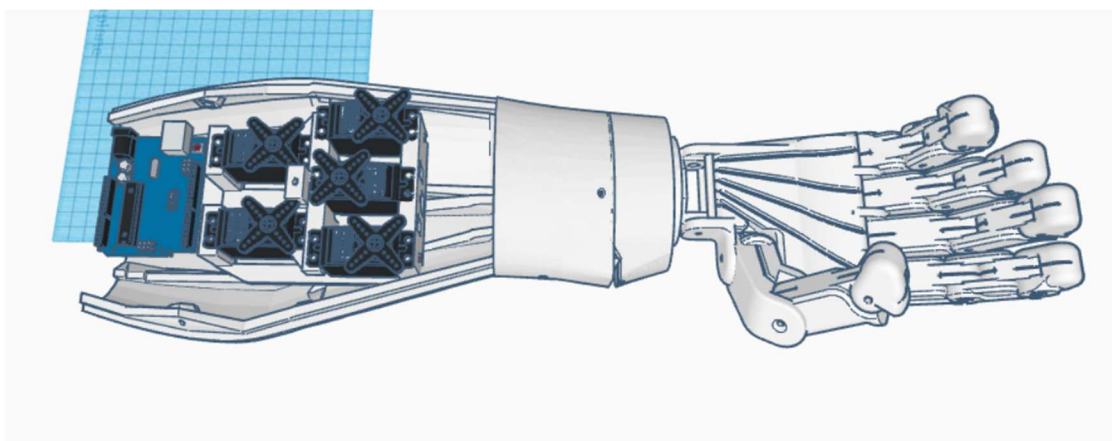


Figura 21 – Braço 3D

A figura seguinte ilustra as medidas do tamanho do braço no desenho 3D.

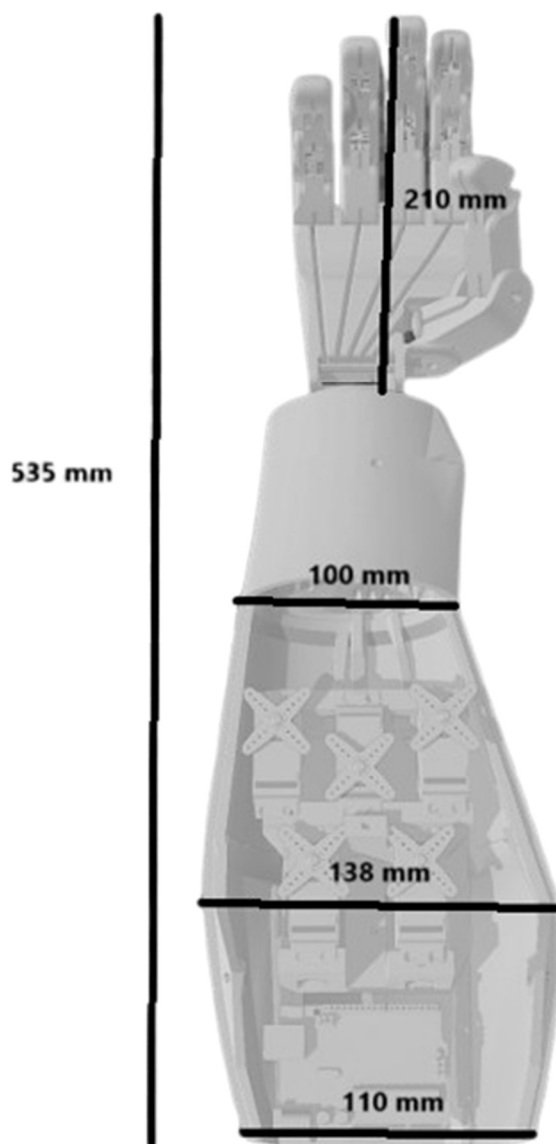


Figura 22 – Medidas do braço 3D

3.1.5 Instalação do Software

3.1.5.1 Processing

Para a instalação e configuração do *Processing* deve-se começar por seleccionar o sistema operativo (Windows, Linux ou MacOS). Em seguida aceder ao site oficial do Processing e fazer o download do executável.

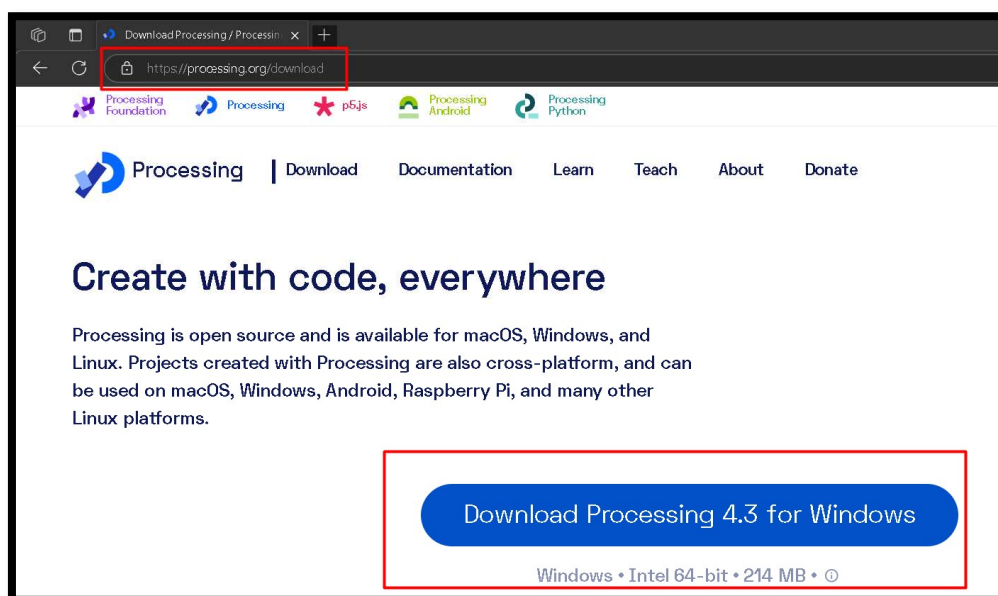


Figura 23 – Acesso ao site oficial do Processing

Após a conclusão do *download* é necessário fazer a extração da pasta.



Figura 24 – Extração da Pasta Zipada

3.1.5.2 Arduino

Na instalação e configuração do Arduino, começa-se por visitar o site oficial do Arduino e descarregar o ficheiro executável.

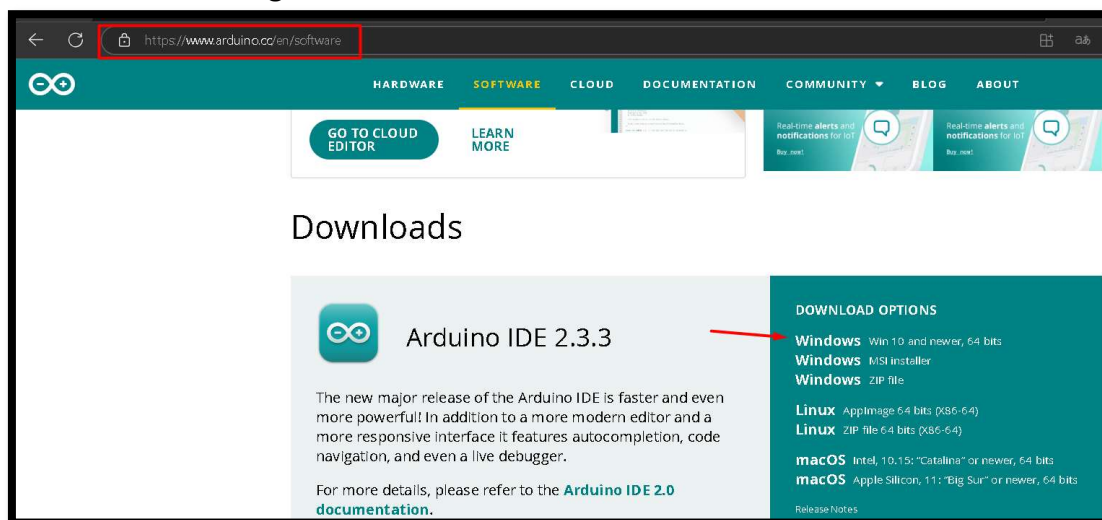


Figura 25 – Instalação do Arduino

Em seguida, executa-se e inicia-se a configuração da pasta onde se encontram localizados os ficheiros da aplicação. Realizada a operação está pronto a ser utilizado.

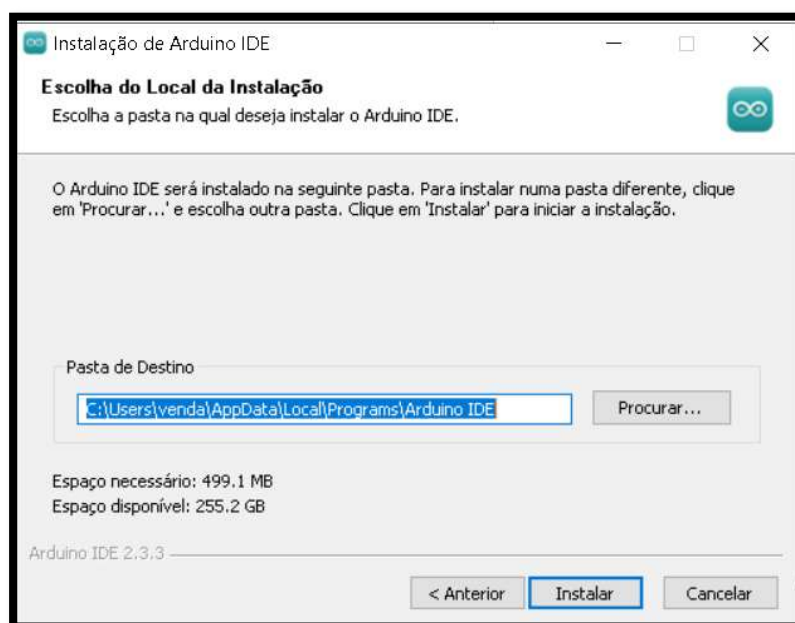


Figura 26 – Configuração da localização da aplicação

3.2 Fase 2 – Impressão 3D das Peças do Braço Robótico

3.2.1 Preparação dos ficheiros de impressão 3D das peças do braço

Para realizar a impressão 3D começa-se por fazer o *download* dos STL. O site utilizado foi o Inmoov que é um site com o foco na criação de um robô. Os stl são grátis possibilitando que qualquer utilizador os possa utilizar.

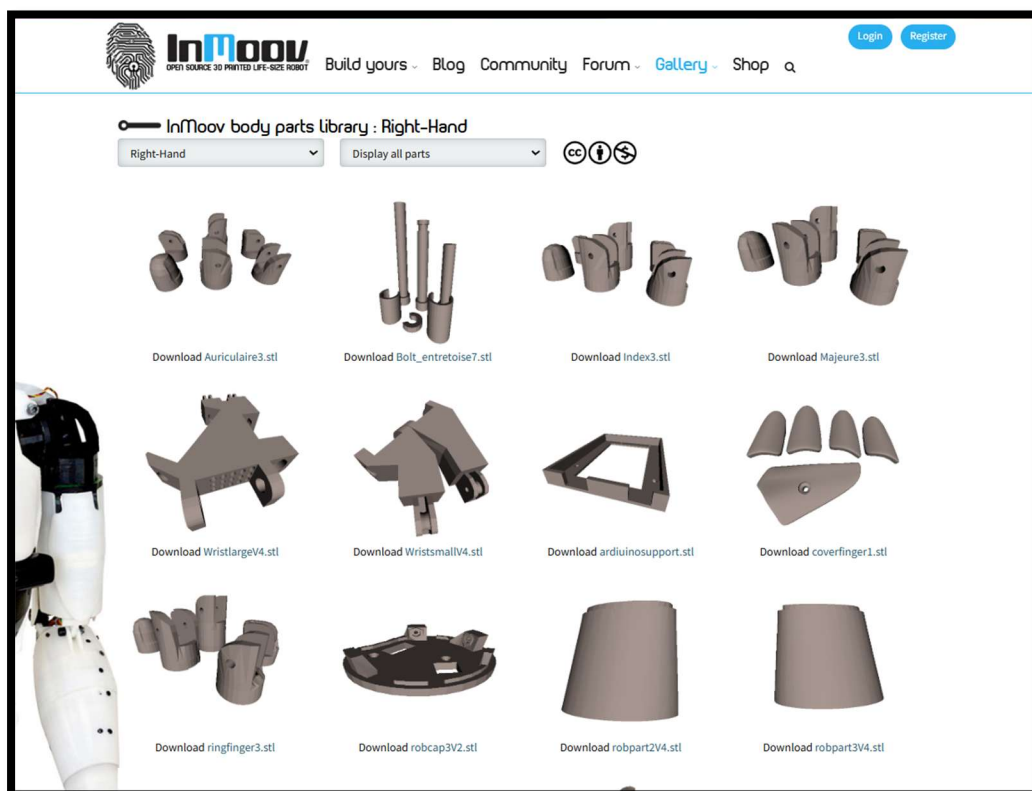


Figura 27 – Aquisição dos STL

Curiosidade: Os arquivos STL do InMoov são modelos tridimensionais usados para imprimir as peças do robô InMoov em impressoras 3D. Os arquivos fazem parte do projeto InMoov, um robô humanóide de código aberto criado por Gaël Langevin.

3.2.2 Impressão das peças utilizando filamento adequado

Para fazer a impressão das peças utilizam-se os serviços de uma empresa. São enviados os STL do braço e após alguns dias obtém-se a impressão.



Figura 28 – Pedido de Impressão

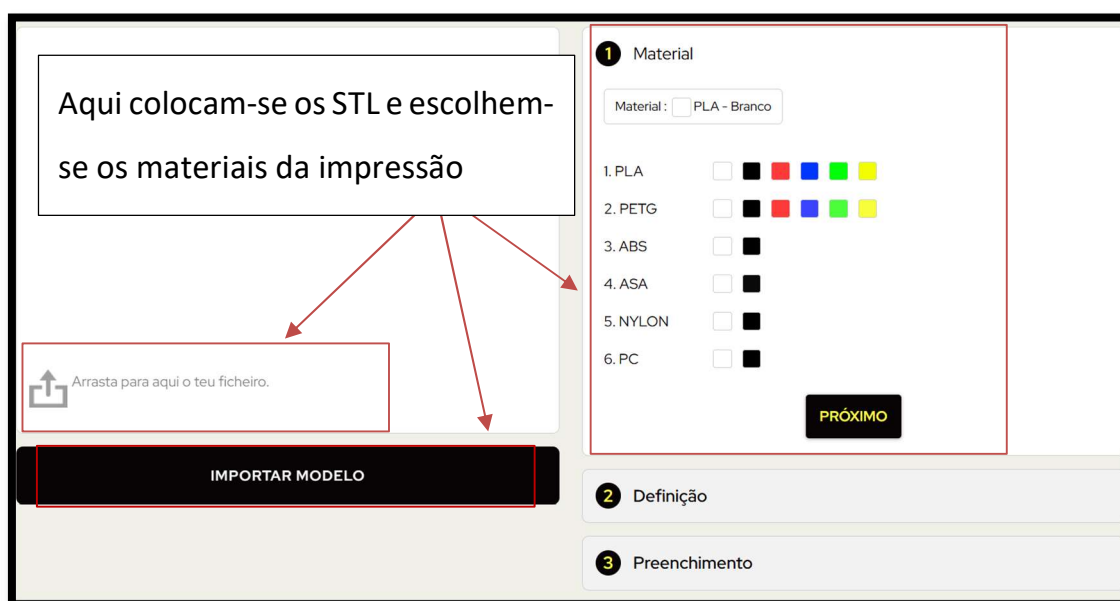


Figura 29 – Envio do STL

3.3 Fase 3 – Montagem Mecânica

Nesta fase faz-se a montagem de todas as peças impressas para criar o braço robótico funcional.

Após a elaboração da impressão, inicia-se a montagem do antebraço.



Figura 30 – Perspetiva do antebraço

Colocam-se os servo motores na estrutura 3D.

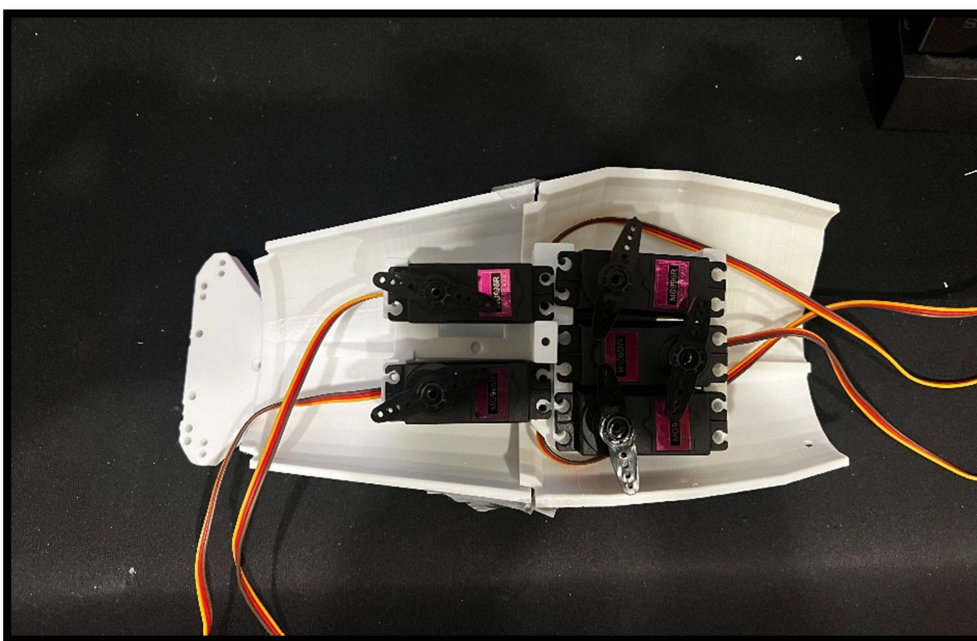


Figura 31 – Montagem antebraço

Colocam-se as peças por cima do servo que é para puxar o fio.

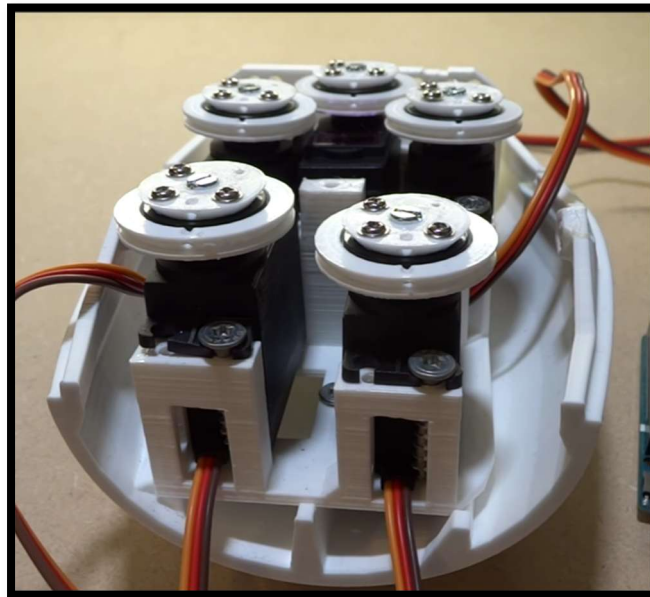


Figura 32 – Montagem dos Servos

Monta-se a mão e passam-se todos os fios.



Figura 33 – Montagem da Mão

Colocam-se os acabamentos dos dedos, colando as últimas peças.

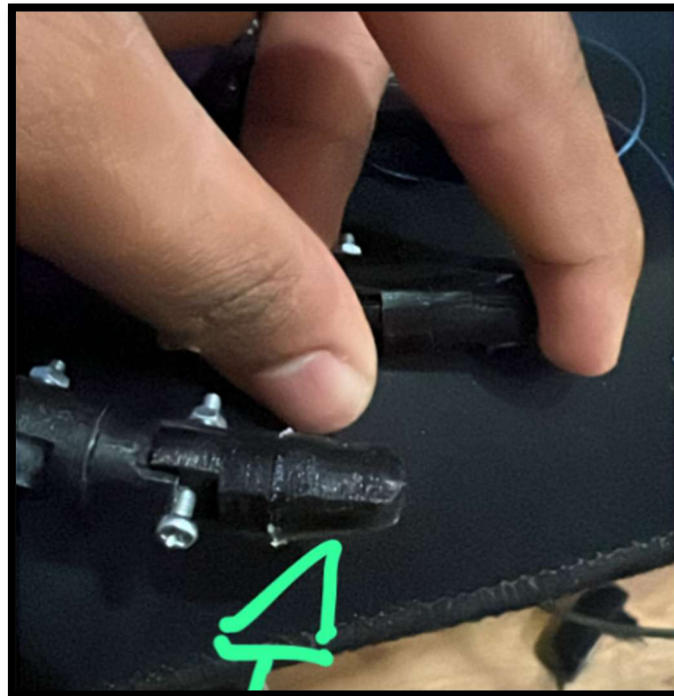


Figura 34 – Acabamento

Juntam-se a mão ao antebraço e faz-se a ligação dos fios no servo.

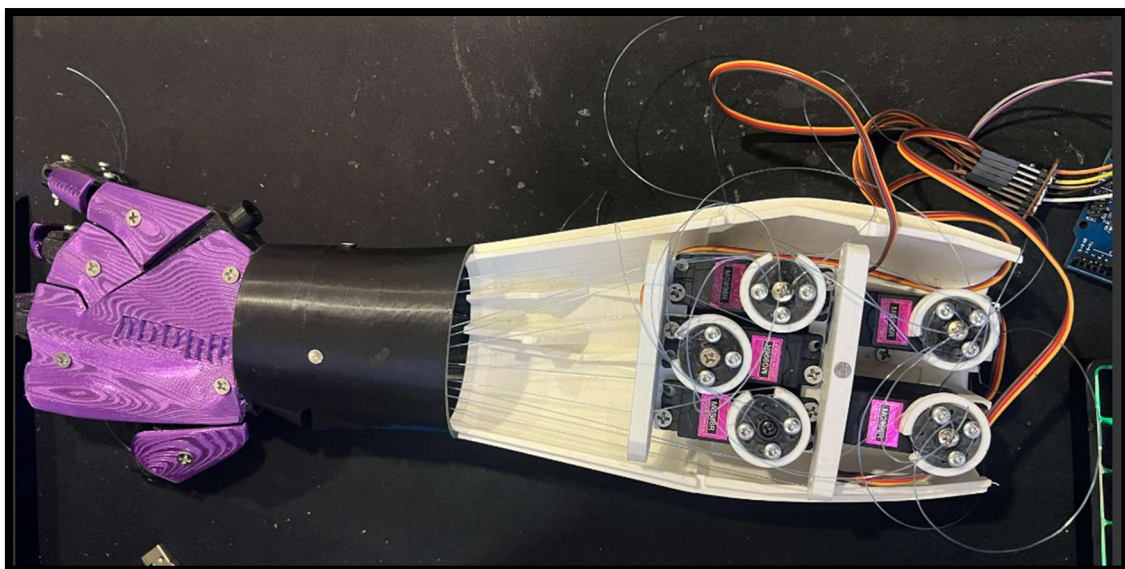


Figura 35 – Finalização da montagem do Braço

3.4 Fase 4 – Montagem do Circuito Eletrónico

3.4.1 Montagem e ligação de todos os componentes eletrónicos

O processo da montagem foi realizado de forma sequencial. Inicialmente, ligaram-se os servos motores ao microcontrolador, e testado o seu funcionamento. De seguida, ligou-se o *Leap Motion*, assim como o microcontrolador ao computador para ver de estava tudo de acordo com o circuito planeado.

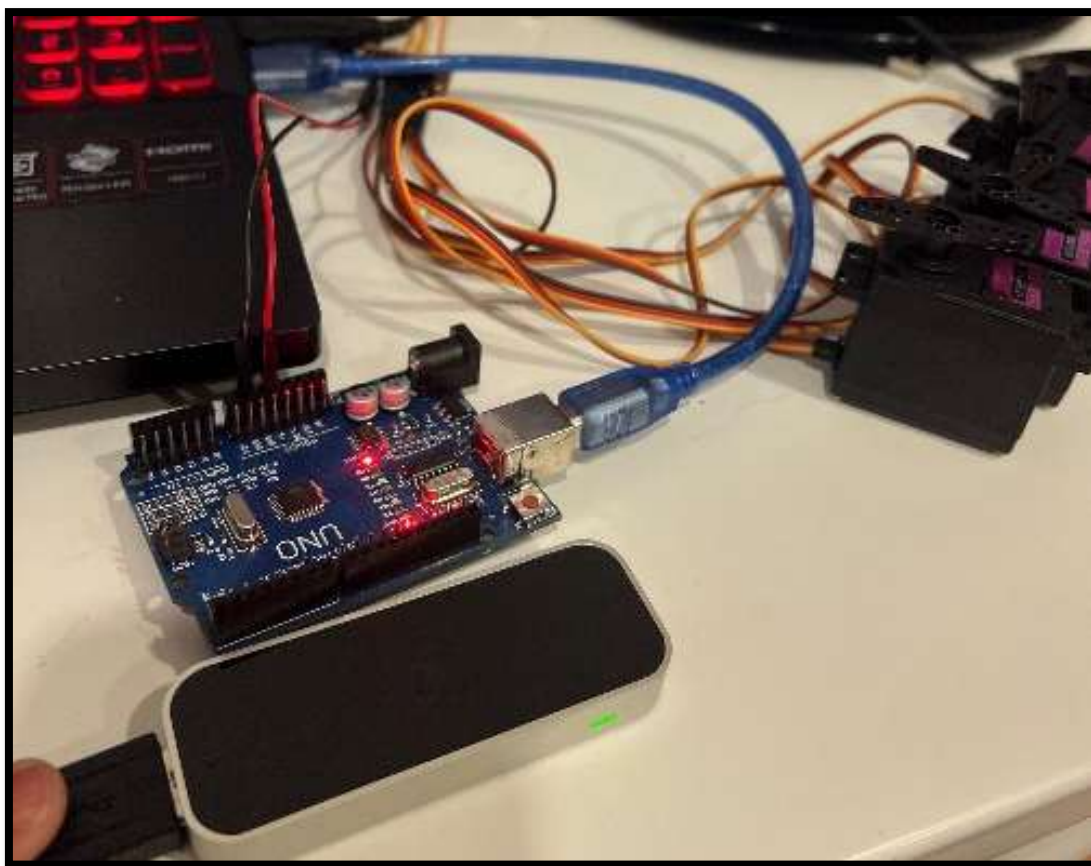


Figura 36 – Montagem do circuito

Na etapa seguinte, fez-se a ligação no protótipo do braço robótico.

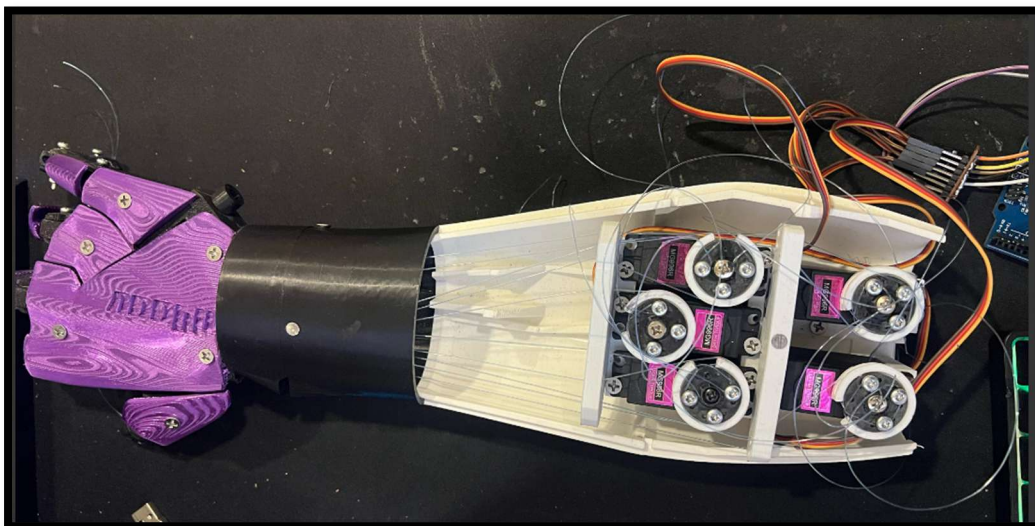


Figura 37 – Ligação no protótipo

Por último, realizou-se o teste e funcionamento das ligações no protótipo.

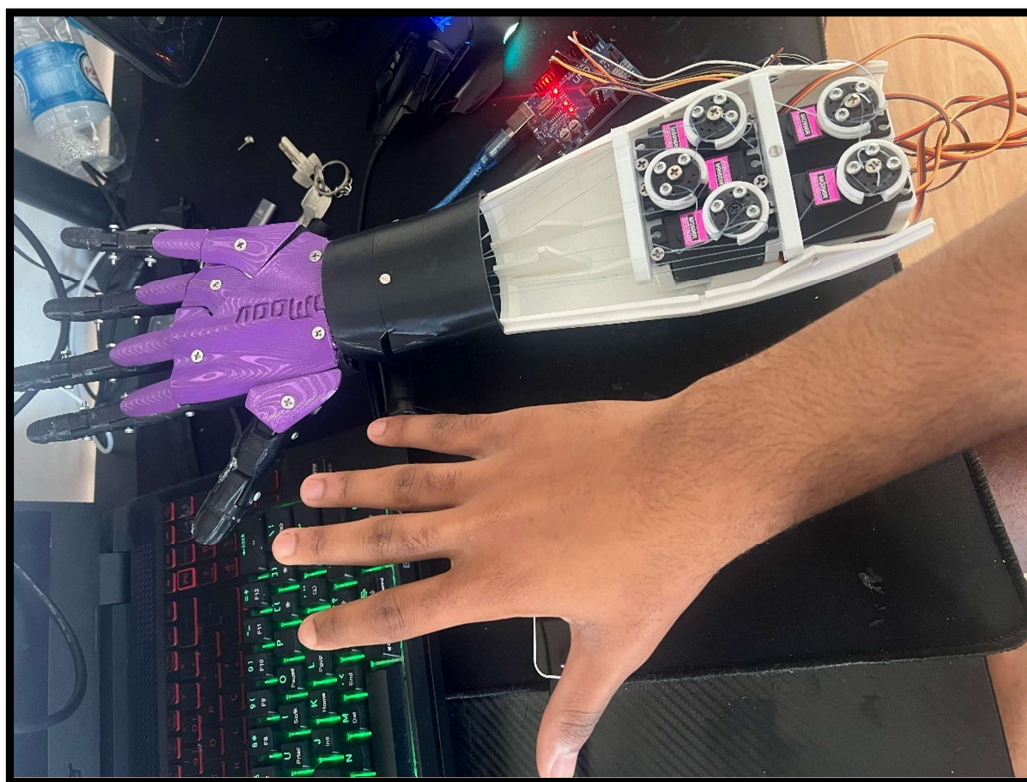


Figura 38 – Teste das ligações

3.4.2 Teste de todas as ligações

Para garantir que todas as ligações estão seguras e funcionais, é necessário certificar-se que as ligações criadas entre os servos motores e o arduino estão corretas, verificando e testando caso exista alguma falha na ligação.

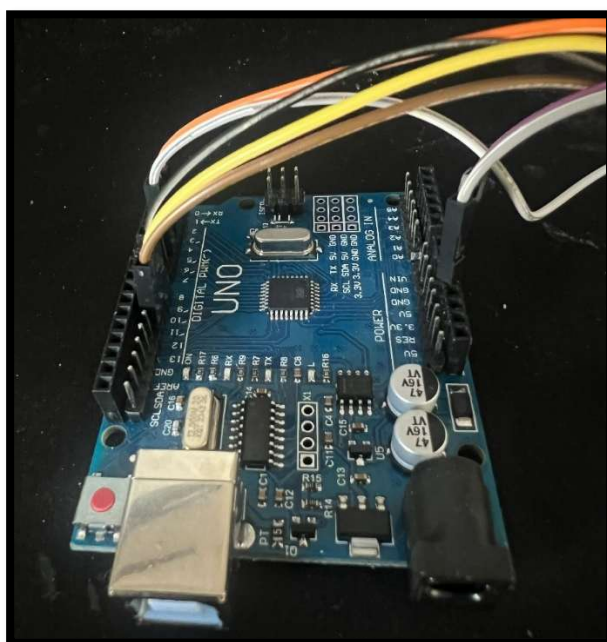


Figura 39 – Ligação Arduino



Figura 40 – Ligação servo motor

3.5 Fase 5 – Programação do Microcontrolador Arduino

3.5.1 Aquisição de dados

Na fase inicial, é necessário programar a aquisição de dados utilizando o *Leap Motion* que envia os dados para o Processing.

No processo de programação, colocam-se as bibliotecas da porta serial para a comunicação do Processing e a biblioteca do *Leap Motion*. Desta forma, é possível a obtenção dos dados.

```
// Importa as bibliotecas necessárias
import controlP5.*; // Interface gráfica (ListBox)
import de.voidplus.leapmotion.*; // Controlo do Leap Motion
import processing.serial.*; // Comunicação Serial (com Arduino)
```

Código 1 – Bibliotecas

De seguida, declaram-se as variáveis, para os dados do *Leap Motion*, lista de portas COM e ControlP5 para criar a interface gráfica de uma tabela.

```
// Declaração de variáveis
LeapMotion leap; // Objeto do Leap Motion
ControlP5 cp5; // Interface gráfica
String[] serialPorts; // Lista de portas seriais disponíveis
ListBox portList; // ListBox para escolha da porta
Serial porta; // Porta Serial realmente usada para
comunicação
```

Código 2 – Variáveis

No **void setup** começa-se por definir o tamanho da tela, a cor de fundo e inicia-se as bibliotecas do ControlP5.

```
void setup() {
  size(800, 500, P3D); // Define o tamanho da tela e o renderizador 3D
  background(255); // Cor de fundo inicial (branco)
  cp5 = new ControlP5(this); // Inicializa a biblioteca ControlP5
```

Código 3 – Void Setup

Cria-se uma **listbox** usando o `controlP5`. Esta lista seleciona as portas COM disponíveis, definindo também o tamanho e a posição em que estaria a lista.

```
// Cria uma ListBox para selecionar a porta serial
portList = cp5.addListBox("portList")
    .setPosition(0, 0)      // Posição na tela
    .setSize(100, 150)     // Tamanho da caixa
    .setItemHeight(20)     // Altura dos itens
    .setBarHeight(20)      // Altura da barra do topo
    .setLabel("Selecione a Porta"); // Texto do título
```

Código 4 – ListBox

Na etapa seguinte, temos uma função para preencher a `listbox` com as portas series disponíveis, usando as portas guardadas nas variáveis antes definidas `serialPort`.

```
// Preenche a ListBox com as portas seriais disponíveis
serialPorts = Serial.list();
for (int i = 0; i < serialPorts.length; i++) {
    portList.addItem(serialPorts[i], i);
}
// Inicializa o Leap Motion
leap = new LeapMotion(this);
}
```

Código 5 – Listagem serial port

Uma vez preenchida a `listbox`, cria-se uma função que a partir do momento que a porta é selecionada entra em funcionamento e se o utilizador escolher uma outra porta, a porta serial atual é fechada e a outra porta é iniciada.

```
// Função chamada automaticamente quando o utilizador seleciona uma
// porta na ListBox
void portList(int n) {
    String portName = serialPorts[n]; // Obtém o nome da porta
    // selecionada
    if (porta != null) {
        porta.stop(); // Fecha a porta atual se já estiver aberta
    }
    porta = new Serial(this, portName, 9600); // Abre a nova porta a
    // 9600 bps
    println("Porta selecionada: " + portName);
}
```

Código 6 – Iniciar porta

Para o **void draw**, começa-se por colocar a atualização da tela a cada quadro, obtendo a taxa de quadros do *Leap Motion*. Faz-se a deteção das mãos para desenhar a sua representação, capturam-se os movimentos de todos os dedos da mão e cria-se um array para a guardar o ângulo de todos eles.

```
void draw() {  
    background(255); // Limpa a tela a cada quadro  
  
    int fps = leap.getFrameRate(); // Pega a taxa de quadros do Leap Motion  
  
    // Percorre todas as mãos detectadas pelo Leap Motion  
    for (Hand hand : leap.getHands()) {  
        hand.draw(); // Desenha a representação da mão  
  
        // Captura todos os dedos da mão  
        Finger[] fingers = hand.getFingers().toArray(new Finger[0]);  
  
        // Cria um array para armazenar os ângulos dos dedos  
        float[] fingerAngles = new float[5];  
    }  
}
```

Código 7 – Void draw

O fragmento de código seguinte descreve a necessidade de que se percorram todos os dedos para obter a posição de cada dedo, a direção para onde o dedo está apontando e um cálculo do ângulo com base num vector (array). Esse vector permite que os dados do dedo sejam mais precisos. Após o cálculo dos ângulos, são guardados os valores no array.

```
// Percorre todos os dedos  
for (int i = 0; i < fingers.length; i++) {  
    Finger finger = fingers[i];  
    // Obtém a posição da ponta do dedo  
    PVector fingerTip = finger.getPosition();  
    // Obtém a direção do dedo  
    PVector fingerDirection = finger.getDirection();  
  
    // Calcula o ângulo do dedo em relação a um vetor (1, 1, 1) - simplificação  
    float angle = degrees(PVector.angleBetween(fingerDirection,  
new PVector(1, 1, 1)));  
    fingerAngles[i] = angle; // Salva o ângulo no array  
}
```

Código 8 – Cálculo do Ângulo

Na etapa seguinte, cria-se uma pequena esfera vermelha na ponta de cada dedo, para facilitar o acompanhamento do utilizador. Depois, é chamada a função para enviar os dados para o arduino.

```
// Desenha uma pequena esfera na ponta do dedo
fill(255, 0, 0); // Vermelho
noStroke();      // Sem contorno
pushMatrix();
translate(fingerTip.x, fingerTip.y, fingerTip.z);
sphere(5); // Esfera de raio 5
popMatrix();
}
// Após calcular os ângulos de todos os dedos, envia para o
Arduino
EnviarAngulo(fingerAngles);
}
}
```

Código 9 – Enviar Ângulo

Nesta etapa, cria-se uma função para enviar os dados para o arduino pela porta serial. A função garante que a porta esteja aberta e faz o mapeamento dos dados para uma precisão maior e garante que os valores fiquem exatamente dentro do intervalo de 0 a 180 que são os valores do ângulo mínimo e máximo dos servos motores.

```
// Função que formata e envia os ângulos para o Arduino via Serial
void EnviarAngulo(float[] angles) {
    if (porta == null) return; // Garante que a porta esteja aberta
    float mappedAngles[] = new float[5];
    // Mapeia os ângulos capturados para valores entre 0 e 180 graus
    mappedAngles[0] = map(angles[0], 100, 170, 0, 180); // Polegar
    mappedAngles[1] = map(angles[1], 30, 150, 0, 180); // Indicador
    mappedAngles[2] = map(angles[2], 30, 150, 0, 180); // Médio
    mappedAngles[3] = map(angles[3], 0, 140, 0, 180); // Anelar
    mappedAngles[4] = map(angles[4], 25, 130, 0, 180); //
Míndinho
    // Garante que os valores fiquem exatamente dentro do intervalo [0,
    180]
    for (int i = 0; i < 5; i++) {
        mappedAngles[i] = constrain(mappedAngles[i], 0, 180);
    }
}
```

Código 10 – Início da função

Na fase final, preparam-se os dados para transmitir. Envia-se os dados dos dedos separados por vírgulas, e uma quebra de linha. Faz-se os dados aparecer no console e enviam-se os dados pela porta serial para o arduino.

```
// Prepara uma string para enviar os dados (formato:
90,45,30,60,20\n)
String data = "";
for (int i = 0; i < mappedAngles.length; i++) {
    data += int(mappedAngles[i]); // Converte para inteiro
    if (i < mappedAngles.length - 1) data += ","; // Se não for
o último, adiciona vírgula
}
data += "\n"; // Finaliza com uma quebra de linha

println("Enviando: " + data); // Mostra no console para
depuração
porta.write(data); // Envia os dados pela Serial
}
```

Código 11 – Enviar para o Arduino

3.5.2 Controlo dos movimentos do braço

Para o controlo dos movimentos do braço, inicia-se a declaração das bibliotecas dos servomotores e a definição dos servos.

```
//Biblioteca dos servomotores
#include <Servo.h>
//Definindo os nomes dos servomotores
Servo DedaoServo;
Servo indicadorServo;
Servo medioServo;
Servo anelarServo;
Servo mindinhoServo;
```

Código 12 – Bibliotecas

De seguida, coloca-se cada servo num pino pwm.

```
// Pinos dos servos
const int DedaoPin = 10;
const int indicadorPin = 3;
const int medioPin = 5;
const int anelarPin = 9;
const int mindinhoPin = 11;
```

Código 13 – PinosPWM

No **void setup** começa-se por definir a velocidade da comunicação da porta serial e colocam-se todos os servos com um ângulo inicial de 90 graus.

```
void setup() {  
  //velocidade de comunicação da porta serial  
  Serial.begin(9600);  
  DedaoServo.write(90);  
  indicadorServo.write(90);  
  medioServo.write(90);  
  anelarServo.write(90);  
  mindinhoServo.write(90);  
}
```

Código 14 – Void Setup Arduino

De seguida, anexam-se os servos aos pinos anteriormente definidos.

```
// Anexa os servos aos pinos  
DedaoServo.attach(DedaoPin);  
indicadorServo.attach(indicadorPin);  
medioServo.attach(medioPin);  
anelarServo.attach(anelarPin);  
mindinhoServo.attach(mindinhoPin);
```

Código 15 – Anexar pinos

Neste fragmento de código começam-se por lê as strings enviadas pelo Processing até à quebra de linha(/n). Os dados são guardados num array. De seguida, cria-se uma função para dividir a string recebida em valores individuais. Após esse processo, é necessário converter as string em tokens e de tokens para inteiros e só depois guarda-se tudo num array.

```
void loop()  
{  
  if (Serial.available() > 0) {  
    // Lê a string enviada pelo Processing  
    String DadosDedos = Serial.readStringUntil('\n');  
    // Array para armazenar os valores recebidos  
    int meuArray[5];  
    int i = 0;  
    // Divide a string recebida em valores individuais  
    char* Ang = strtok((char*)DadosDedos.c_str(), ",");  
    while (Ang != NULL) {  
      meuArray[i++] = atoi(Ang); // Converte o token para inteiro  
      // e armazena no array  
      Ang = strtok(NULL, ",");  
    }  
  }  
}
```

Código 16 – Função ler dados

Por último, guardam-se os dados dos ângulos num array e executam-se no servo motor.

```
int Dedo1 = meuArray[0];  
int Dedo2 = meuArray[1];  
int Dedo3 = meuArray[2];  
int Dedo4 = meuArray[3];  
int Dedo5 = meuArray[4];  
DedaoServo.write(Dedo1);  
indicadorServo.write(Dedo2);  
medioServo.write(Dedo3);  
anelarServo.write(Dedo4);  
mindinhoServo.write(Dedo5);  
}  
}
```

Código 17 – Execução no servo motor

3.5.3 Teste e aprimoramento da programação

Nesta fase é testado o funcionamento do Arduino para verificar se existe algum problema nas conexões com os servos ou na codificação.

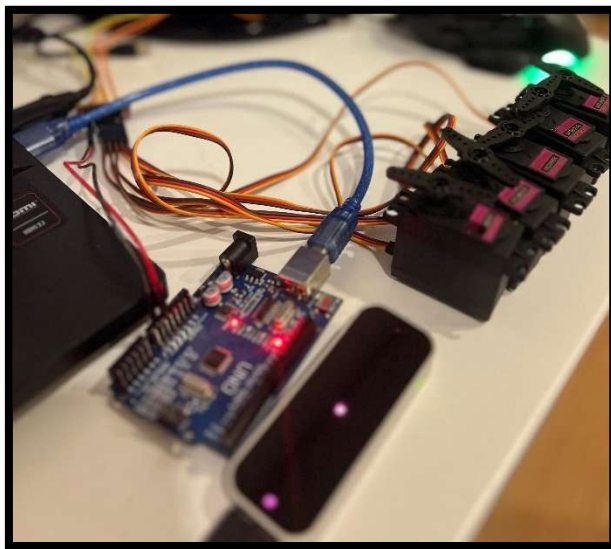


Figura 41 – Teste no Arduino

Nos testes iniciais começa-se por captar os movimentos dos dedos pelo Processing.

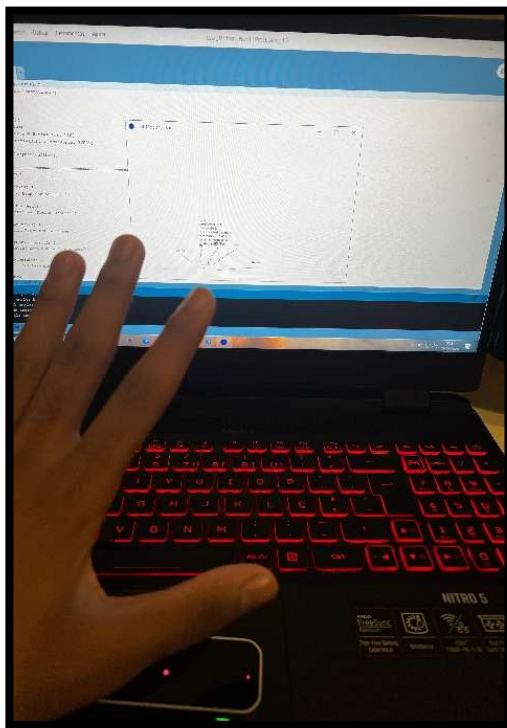


Figura 42 – Teste do programa

A Figura seguinte mostra os servos ligados e funcionando de acordo com o Processing.

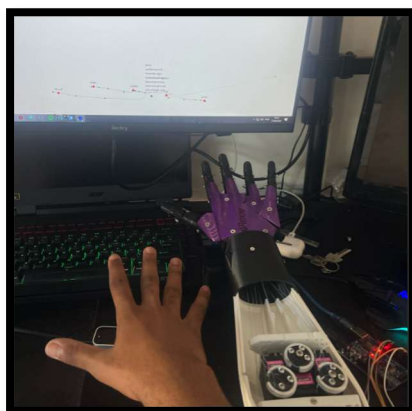


Figura 43 – Teste do Processing

3.6 Fase 6 – Testes e Ajustes finais

3.6.1 Testar o funcionamento completo do braço robótico

Para o teste de funcionamento completo do braço robótico, fez-se a validação da captação de dados e de transmissão, testando o funcionamento do *Leap Motion* que envia dados para o computador que os processa e envia os ângulos para o Arduino fazer com que os servos se movimentem.

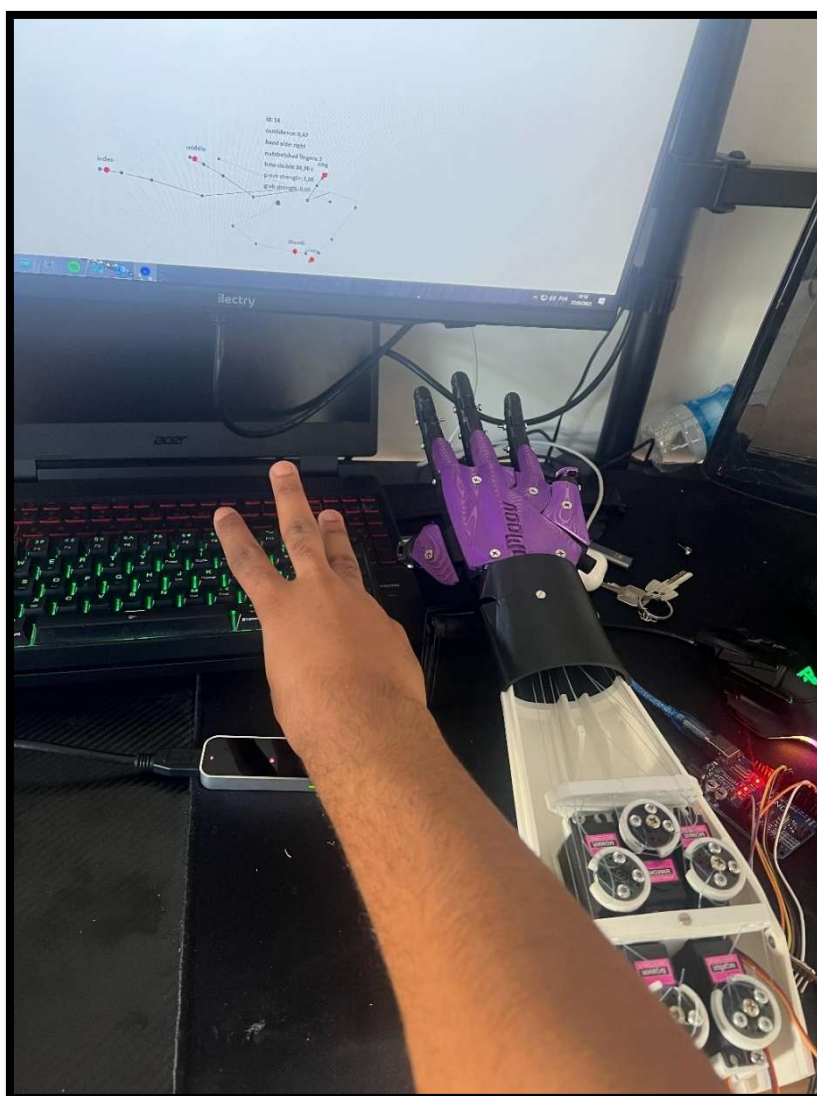


Figura 44 – Teste no braço completo

3.6.2 Realizar ajustes necessários na montagem e programação

Por forma a garantir um desempenho eficiente é necessário realizar alguns ajustes. Os ajustes necessários para o projeto aplicam-se na programação dos dados do ângulo, aplicando o mapeamento para os dados obtidos de cada dedo para uma maior precisão.

Como podemos ver o no código abaixo a aplicação do ajuste em questão:

```
// Mapeia os ângulos capturados para valores entre 0 e 180 graus  
mappedAngles[0] = map(angles[0], 100, 170, 0, 180); // Polegar  
mappedAngles[1] = map(angles[1], 30, 150, 0, 180); // Indicador  
mappedAngles[2] = map(angles[2], 30, 150, 0, 180); // Médio  
mappedAngles[3] = map(angles[3], 0, 140, 0, 180); // Anelar  
mappedAngles[4] = map(angles[4], 25, 130, 0, 180); // Mínimo
```

Código 18 – Ajustes Necessários

Aplicando o ajuste, os dados ficam mais precisos e com melhor representação.

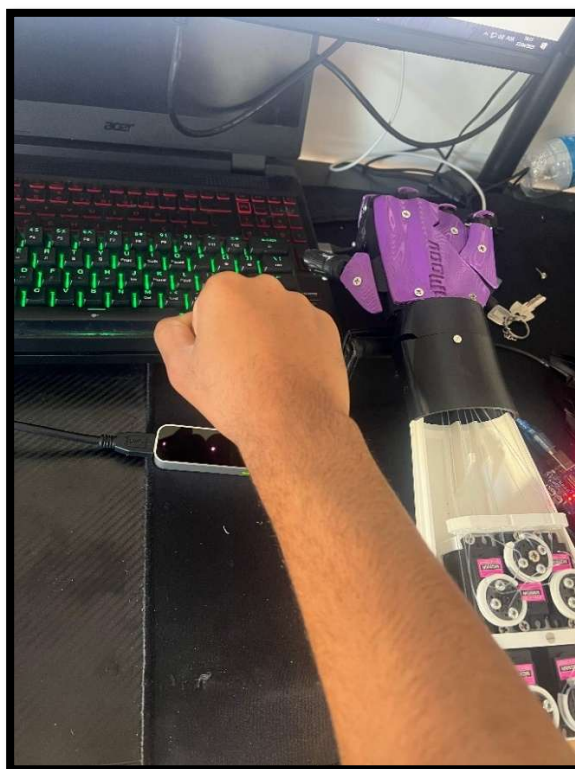


Figura 45 – Ajustes necessários

4 Conclusões

O projeto de desenvolvimento do Braço Robótico foi concluído com sucesso, apesar de alguns contratempos. Um dos maiores obstáculos foi a programação no *Processing* para a aquisição dos ângulos dos dedos a partir do *Leap Motion*. A escassez de informações sobre o uso do *Leap Motion* exigiu uma abordagem proativa e iterativa na resolução de problemas, mas demonstrou-se capacidade de adaptação e resolução de problemas, superando dificuldades menores sem atrasos significativos no cronograma. O êxito alcançado serve como um forte incentivo para futuros projetos, fortalecendo a nossa confiança na capacidade de enfrentar e superar novos desafios com criatividade e determinação.

4.1 Objetivos realizados

O projeto de desenvolvimento do Braço Robótico foi concluído com êxito. No percurso deparámo-nos com desafios que tivemos que superar, sendo um deles a programação do *Processing*, mais propriamente, a captação dos ângulos dos dedos, tendo que aprender diversas formas de lidar com os dados que eram obtidos pelo *Leap Motion*. Inicialmente, nos primeiros contactos com o *Leap Motion*, por ser uma temática não muito abordada em sala de aula, deu-se conta que os mínimos detalhes eram sempre bem-vindos, fazendo repensar a melhor estratégia e abordagem para resolver as dificuldades que iam surgindo.

Não obstante e apesar dessas dificuldades que foram resolvidas com a ajuda dos professores e de forma autónoma, conseguiram-se cumprir todos os objetivos delineados inicialmente.

4.2 Limitações e trabalho futuro

A principal limitação na realização do projeto foi a falta de informação relacionada ao uso do *Leap Motion* para controlo de um Braço Robótico. Foi necessária muita pesquisa para poder ter um pouco de informação que pudesse ser útil. Com essa limitação superada, deu-se seguimento à fase dos testes e lidar com os desafios que surgiram pela frente.

Como trabalho futuro seria interessante pensar por exemplo o controlo gestual de um braço robótico (movimentos da mão no ar, sem contacto físico), a criação de uma interface educativa com controlo por gestos (navegação em menus ou realizar operações apenas com o movimento das mãos), jogos educativos (desenvolver jogos simples que utilizem gestos para interação – tocar em, alvos, puzzles, etc.).

4.3 Apreciação final

No desenvolvimento deste projeto alcançou-se um vasto conhecimento de distintas temáticas, sendo um projeto enriquecedor que possuía vários desafios que foram superados gradualmente, e sem fugir à ideia inicial da criação do Braço Robótico.

No processo da montagem do Braço Robótico, utilizaram-se os vastos conteúdos aprendidos no curso, não apenas sobre programação, mas também como superar os desafios. A concretização de um sistema funcional, que é capaz de captar e replicar movimentos da mão em tempo real, através do uso do *Leap Motion* e do Arduino, demonstra não apenas a efetividade do projeto, mas também a capacidade de enfrentar desafios, investigar soluções e tomar decisões técnicas. É portanto, uma apreciação final muito positiva.

4.4 Autoavaliação

4.4.1 Lounie Costa

Com a conclusão da PAP, consegui atingir os objetivos que tinha definido para o desenvolvimento deste projeto. Apliquei com sucesso os conhecimentos que adquiri ao longo do curso para atender às exigências do projeto de maneira eficaz. Até a data de entrega do relatório, não tenho nenhum módulo ou UFCD em atraso, priorizando o desenvolvimento eficiente do projeto, cumprindo as minhas responsabilidades académicas durante todo o curso. Com toda a dedicação e esforço na realização do projeto, considero que minha classificação seja de 17 valores.

4.4.2 Mário Moniz

Consegui cumprir todos os objetivos definidos para a PAP e colocar em prática e ao mesmo tempo aprofundar a maioria dos conhecimentos adquiridos ao longo do curso. No início do terceiro ano do curso, não estava tão focado nas aulas como deveria, e acabei por deixar alguns módulos em atraso, não por falta de conhecimentos dos conteúdos desses módulos mas pela falta de assiduidade. Consegui superar essa fase e erguer-me pois tinha um objetivo traçado e tinha-o que o concluir. Desistir não é uma opção! Consegui alcançar todos os objetivos do curso, contando com o sucesso da realização da PAP e a finalização de todas as disciplinas. Considero que minha nota final resultante do desenvolvimento da PAP deva ser de 17 valores.

4.5 Formação em Contexto de Trabalho

4.5.1 Lounie Costa

A minha **primeira Formação em Contexto de Trabalho (FCT)** decorreu na empresa **Aquário Eletrónica**, uma empresa bem conhecida no Porto, que se destaca nacionalmente. São especialistas na venda de soluções eletrónicas e de consumo. Durante o meu tempo na empresa, desempenhei várias funções, como repor produtos na loja, ajudar no armazém, separar, picar e embalar encomendas para entrega, além de gerir requisições e transferências entre as lojas. Esta experiência permitiu-me desenvolver competências técnicas ligadas a produtos eletrónicos, atendimento ao cliente e logística, além de aprimorar as minhas capacidades interpessoais, como trabalho em equipa, organização e responsabilidade profissional.

A segunda FCT iniciada no dia 22 de abril, consiste na elaboração de uma **prática simulada** na **Escola Europeia de Ensino Profissional (EEEP)** para o desenvolvimento de um projeto de um site. O projeto tem como objetivo a criação de um site para fazer a gestão de estágios curriculares, permitindo a facilidade na gestão dos estágios.



4.5.2 Mário Moniz

A minha primeira FCT foi realizada na **Cruz Vermelha de Portugal**, localizada em Braga. Trabalhei no departamento de informática da Cruz Vermelha, desenvolvendo tarefas como a resolução de Tickets, instalação e configuração de Firewalls, gestão e resolução de problemas de impressoras, formatação de máquinas, gestão de servidores web, etc.

A segunda FCT foi realizada na **Aquário Eletrónica**, localizada no Porto, o estágio foi bastante interessante e produtivo. Tinha como funções a resolução de problemas de impressoras, apoio ao cliente, embalamento de encomendas, requisição de stock. Esta experiência permitiu-me desenvolver competências técnicas ligadas a produtos eletrónicos, atendimento ao cliente e logística, além de aprimorar minhas capacidades interpessoais, como trabalho em equipa, organização e responsabilidade profissional.



5 Bibliografia

1. Morawiec, Darius. *github. leap-motion-processing*. [Online] [Citação: 10 de Dezembro de 2024.] <https://github.com/nok/leap-motion-processing>.
2. Ultraleap. *Leap Motion Controller*. [Online] [Citação: 10 de Dezembro de 2024.] <https://leap2.ultraleap.com/downloads/leap-motion-controller/>.
3. MyRobotLab. [Online] [Citação: 20 de janeiro de 2025.] <https://myrobotlab.org/content/get-fingers-position-leapmotion>.
4. how to get the palm, finger position in Leap Motion. *Unit*. [Online] [Citação: 18 de janeiro de 2025.] <https://discussions.unity.com/t/how-to-get-the-palm-finger-position-in-leap-motion/537229>.
5. Langevin, Gael. Inmoov. [Online] [Citação: 7 de Novembro de 2024.] <https://inmoov.fr/>.
6. Gael, Langevin. Hand and Forarm. *inmoov*. [Online] [Citação: 25 de janeiro de 2025.] <https://inmoov.fr/hand-and-forarm/>.
7. XanderDIY. Inmoov hand and forarm (STEP BY STEP tutorial). *youtube*. [Online] [Citação: 7 de Maio de 2019.] https://www.youtube.com/watch?v=4t1daCFQ1OE&ab_channel=XanderDIY.

6 Anexos

A. Código

A1. Arduino

```
#include <Servo.h>
// Define os servos para cada dedo
Servo DedaoServo;
Servo indicadorServo;
Servo medioServo;
Servo anelarServo;
Servo mindinhoServo;
// Pinos dos servos
const int DedaoPin = 10;
const int indicadorPin = 3;
const int medioPin = 5;
const int anelarPin = 9;
const int mindinhoPin = 11;

void setup() {
  // Inicializa a comunicação serial
  Serial.begin(9600);
  DedaoServo.write(90);
  indicadorServo.write(90);
  medioServo.write(90);
  anelarServo.write(90);
  mindinhoServo.write(90);

  // Anexa os servos aos pinos
  DedaoServo.attach(DedaoPin);
  indicadorServo.attach(indicadorPin);
  medioServo.attach(medioPin);
  anelarServo.attach(anelarPin);
  mindinhoServo.attach(mindinhoPin);
}

void loop()
{
  if (Serial.available() > 0) {
    // Lê a string enviada pelo Processing
    String DadosDedos = Serial.readStringUntil('\n');

    // Array para armazenar os valores recebidos
    int meuArray[5]; // Ajuste o tamanho conforme necessário
    int i = 0;

    // Divide a string recebida em valores individuais
    char* Ang = strtok((char*)DadosDedos.c_str(), ",");
```

```
while (Ang != NULL) {
    meuArray[i++] = atoi(Ang); // Converte o token para inteiro
    e armazena no array
    Ang = strtok(NULL, ",");
}
int Dedo1 = meuArray[0];
int Dedo2 = meuArray[1];
int Dedo3 = meuArray[2];
int Dedo4 = meuArray[3];
int Dedo5 = meuArray[4];

DedaoServo.write(Dedo1);
indicadorServo.write(Dedo2);
medioServo.write(Dedo3);
anelarServo.write(Dedo4);
mindinhoServo.write(Dedo5);
}
}
```

A2. Processing

```
// Importa as bibliotecas necessárias
import controlP5.*; // Interface gráfica (ListBox)
import de.voidplus.leapmotion.*; // Controle do Leap Motion
import processing.serial.*; // Comunicação Serial (com Arduino)

// Declaração de variáveis
LeapMotion leap; // Objeto do Leap Motion
ControlP5 cp5; // Interface gráfica
String[] serialPorts; // Lista de portas seriais disponíveis
ListBox portList; // ListBox para escolha da porta
Serial porta; // Porta Serial realmente usada para
comunicação

void setup() {
    size(800, 500, P3D); // Define o tamanho da tela e o renderizador
    3D

    background(255); // Cor de fundo inicial (branco)
    cp5 = new ControlP5(this); // Inicializa a biblioteca ControlP5

    // Cria uma ListBox para seleccionar a porta serial
    portList = cp5.addListBox("portList")
        .setPosition(0, 0) // Posição na tela
        .setSize(100, 150) // Tamanho da caixa
        .setItemHeight(20) // Altura dos itens
        .setBarHeight(20) // Altura da barra do topo
        .setLabel("Selecione a Porta"); // Texto do título

    // Preenche a ListBox com as portas seriais disponíveis
    serialPorts = Serial.list();
    for (int i = 0; i < serialPorts.length; i++) {
        portList.addItem(serialPorts[i], i);
    }

    // Inicializa o Leap Motion
    leap = new LeapMotion(this);
}
```

```
// Função chamada automaticamente quando o usuário seleciona uma
porta na ListBox
void portList(int n) {
    String portName = serialPorts[n]; // Obtém o nome da porta
    selecionada
    if (porta != null) {
        porta.stop(); // Fecha a porta atual se já estiver aberta
    }
    porta = new Serial(this, portName, 9600); // Abre a nova porta a
    9600 bps
    println("Porta selecionada: " + portName);
}
void draw() {

    background(255); // Limpa a tela a cada quadro

    int fps = leap.getFrameRate(); // Pega a taxa de quadros do Leap
    Motion (não usada, mas poderia ser)

    // Percorre todas as mãos detectadas pelo Leap Motion
    for (Hand hand : leap.getHands()) {
        hand.draw(); // Desenha a representação da mão

        // Captura todos os dedos da mão
        Finger[] fingers = hand.getFingers().toArray(new Finger[0]);

        // Cria um array para armazenar os ângulos dos dedos
        float[] fingerAngles = new float[5];

        // Percorre todos os dedos
        for (int i = 0; i < fingers.length; i++) {
            Finger finger = fingers[i];

            // Obtém a posição da ponta do dedo
            PVector fingerTip = finger.getPosition();

            // Obtém a direção do dedo
            PVector fingerDirection = finger.getDirection();
            // Calcula o ângulo do dedo em relação a um vetor (1, 1, 1) -
            simplificação
            float angle = degrees(PVector.angleBetween(fingerDirection,
            new PVector(1, 1, 1)));
            fingerAngles[i] = angle; // Salva o ângulo no array

            // Desenha uma pequena esfera na ponta do dedo
            fill(255, 0, 0); // Vermelho
            noStroke(); // Sem contorno
            pushMatrix();
            translate(fingerTip.x, fingerTip.y, fingerTip.z);
            sphere(5); // Esfera de raio 5
            popMatrix();
        }
        // Após calcular os ângulos de todos os dedos, envia para o
        Arduino
        EnviarAngulo(fingerAngles);
    }
}
```

```
// Função que formata e envia os ângulos para o Arduino via Serial
void EnviarAngulo(float[] angles) {
    if (porta == null) return; // Garante que a porta esteja aberta

    float mappedAngles[] = new float[5];

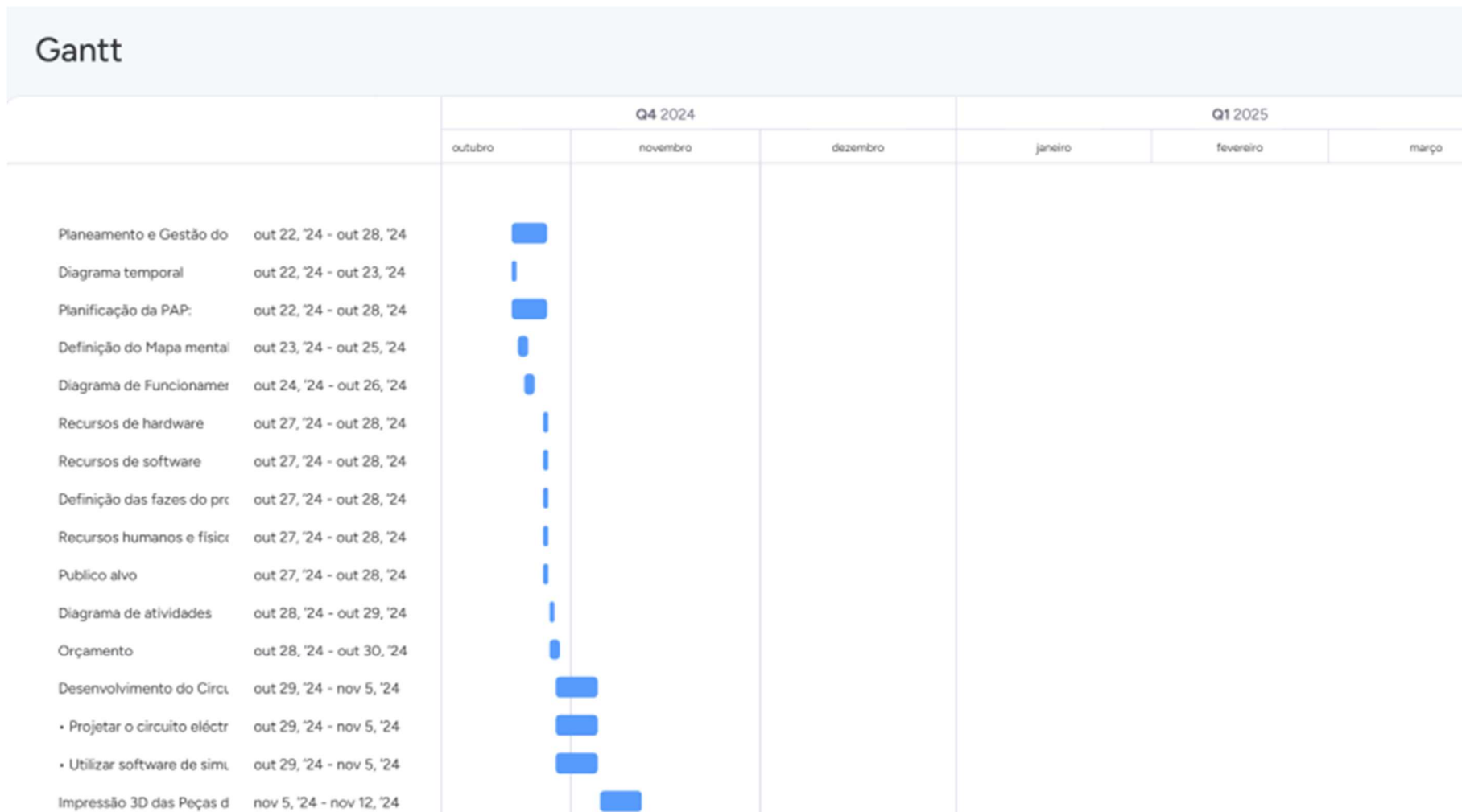
    // Mapeia os ângulos capturados para valores entre 0 e 180
    graus
    mappedAngles[0] = map(angles[0], 100, 170, 0, 180); // Polegar
    mappedAngles[1] = map(angles[1], 30, 150, 0, 180); //
Indicador
    mappedAngles[2] = map(angles[2], 30, 150, 0, 180); // Médio
    mappedAngles[3] = map(angles[3], 0, 140, 0, 180); // Anelar
    mappedAngles[4] = map(angles[4], 25, 130, 0, 180); // Mínimo

    // Garante que os valores fiquem exatamente dentro do intervalo
    [0, 180]
    for (int i = 0; i < 5; i++) {
        mappedAngles[i] = constrain(mappedAngles[i], 0, 180);
    }

    // Prepara uma string para enviar os dados (formato:
    90,45,30,60,20\n)
    String data = "";
    for (int i = 0; i < mappedAngles.length; i++) {
        data += int(mappedAngles[i]); // Converte para inteiro
        if (i < mappedAngles.length - 1) data += ","; // Se não for
o último, adiciona vírgula
    }
    data += "\n"; // Finaliza com uma quebra de linha

    println("Enviando: " + data); // Mostra no console para
depuração
    porta.write(data); // Envia os dados pela Serial
}
```


A. Diagrama Temporal





Braço Robótico

- | | |
|-----------------------------|---------------------------|
| • Preparar os ficheiros de | nov 5, '24 - nov 12, '24 |
| • Imprimir as peças utiliza | nov 5, '24 - nov 12, '24 |
| Instalação e Configuração | nov 12, '24 - nov 19, '24 |
| Montagem Mecânica: | nov 19, '24 - nov 26, '24 |
| • Montar todas as peças ir | nov 19, '24 - nov 26, '24 |
| • Montagem do Circuito E | nov 19, '24 - nov 26, '24 |
| Programação do Microcor | nov 19, '24 - jan 15, '25 |
| • Programar o Arduino par | nov 19, '24 - jan 15, '25 |
| • Montar e ligar todos os c | nov 26, '24 - dez 3, '24 |
| • Garantir que todas as lig | nov 26, '24 - dez 3, '24 |
| • Testar e afinar a program | dez 3, '24 - dez 10, '24 |
| Testes e Ajustes finais: | dez 3, '24 - dez 10, '24 |
| • Testar o funcionamento | dez 3, '24 - dez 10, '24 |

